Python 是一种高级编程语言，以其简洁的语法、强大的功能和广泛的库支持而广受欢迎。它适用于多种编程任务，包括 Web 开发、数据分析、人工智能、自动化脚本等。以下是 Python 的入门讲解，涵盖基础语法、核心概念和一些实用示例。

---

1. Python 的特点

- 简洁易读：Python 的语法简洁，接近自然语言，易于学习和理解。
- 跨平台：Python 可以在多种操作系统上运行，包括 Windows、Linux 和 macOS。
- 丰富的库：Python 拥有庞大的标准库和第三方库，支持各种编程任务。
- 动态类型：Python 是一种动态类型语言，变量的类型在运行时自动推断。
- 可扩展性：Python 可以与 C/C++ 等语言结合，用于性能优化。

---

2. 安装 Python

1. 下载 Python访问 Python 官方网站，下载适合你操作系统的 Python 版本。
2. 安装 Python
   - Windows：运行安装程序，确保勾选“Add Python to PATH”选项。
   - macOS：使用 Homebrew 安装：

```
brew install python
```

   - Linux：使用包管理器安装：

验证安装

打开终端或命令提示符，输入以下命令：

```
python --version
```

如果安装成功，你会看到 Python 版本号。

3. Python 入门

下面的例子以是否显示提示符（>>> 与 ...）区分输入与输出：输入例子中的代码时，要键入以提示符开头的行中提示符后的所有内容；未以提示符开头的行是解释器的输出。注意，例子中的某行出现的第二个提示符是用来结束多行命令的，此时，要键入一个空白行。

你可以通过在示例方块右上角的 `>>>` 上点击来切换显示提示符和输出。 如果你隐藏了一个示例的提示符和输出，那么你可以方便地将输入行复制并粘贴到你的解释器中。

本手册中的许多例子，甚至交互式命令都包含注释。Python 注释以 `#` 开头，直到该物理行结束。注释可以在行开头，或空白符与代码之后，但不能在字符串里面。字符串中的 # 号就是 # 号。注释用于阐明代码，Python 不解释注释，键入例子时，可以不输入注释。

示例如下：

```
# 这是第一条注释
spam = 1  # 而这是第二条注释
          # ... 而这是第三条!
text = "# 这不是注释因为它是在引号之内。"

```

3.1. Python 用作计算器

现在，尝试一些简单的 Python 命令。启动解释器，等待主提示符（`>>>` ）出现。

3.1.1. 数字

解释器像一个简单的计算器：你可以输入一个表达式，它将给出结果值。 表达式语法很直观：运算符 `+`, `-`, `*` 和 `/` 可被用来执行算术运算；圆括号 (()) 可被用来进行分组。 例如:`>>>`

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5  # 除法运算总是返回一个浮点数
1.6

```

整数（如，`2`、`4`、`20` ）的类型是 int，带小数（如，`5.0`、`1.6` ）的类型是 float。本教程后半部分将介绍更多数字类型。

除法运算 (`/`) 总是返回浮点数。 如果要做 floor division 得到一个整数结果你可以使用 `//` 运算符；要计算余数你可以使用 `%`:`>>>`

```
>>> 17 / 3  # 经典除法运算返回一个浮点数
5.666666666666667
>>>
>>> 17 // 3  # 向下取整除法运算会丢弃小数部分
5
>>> 17 % 3  # % 运算返回相除的余数
2
>>> 5 * 3 + 2  # 向下取整的商 * 除数 + 余数
17
```

```

```

Python 用 ** 运算符计算乘方 [1]：>>>

```
>>> 5 ** 2  # 5 的平方
25
>>> 2 ** 7  # 2 的 7 次方
128

```

等号（=）用于给变量赋值。赋值后，下一个交互提示符的位置不显示任何结果：>>>

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900

```

如果变量未定义（即，未赋值），使用该变量会提示错误：>>>

```
>>> n  # 试图访问一个未定义的变量
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined

```

Python 全面支持浮点数；混合类型运算数的运算会把整数转换为浮点数：>>>

```
>>> 4 * 3.75 - 1
14.0

```

交互模式下，上次输出的表达式会赋给变量 _。把 Python 当作计算器时，用该变量实现下一步计算更简单，例如：>>>

```
>>> tax = 12.5 / 100
>>> price = 100.50
>>> price * tax
12.5625
>>> price + _
113.0625
>>> round(_, 2)
113.06
```

```
```

最好把该变量当作只读类型。不要为它显式赋值，否则会创建一个同名独立局部变量，该变量会用它的魔法行为屏蔽内置变量。
除了 int 和 float，Python 还支持其他数字类型，例如 Decimal 或 Fraction。Python 还内置支持 复数，后缀 j 或 J 用于表示虚数（例如 3+5j ）。

3.1.2. 文本

除了数字 Python 还可以操作文本（由 str 类型表示，称为“字符串”）。 这包括字符 "!", 单词 "rabbit", 名称 "Paris", 句子 "Got your back." 等等. "Yay! :)"。 它们可以用成对的单引号 ('...') 或双引号 ("...") 来标示，结果完全相同 [2]。 >>>

```
>>> 'spam eggs'  # single quotes
'spam eggs'
>>> "Paris rabbit got your back :)! Yay!"  # double quotes
'Paris rabbit got your back :)! Yay!'
>>> '1975'  # digits and numerals enclosed in quotes are also strings
'1975'

```

要标示引号本身，我们需要对它进行“转义”，即在前面加一个 \。 或者，我们也可以使用不同类型的引号:>>>

```
>>> 'doesn\'t'  # 使用 \' 来转义单引号...
"doesn't"
>>> "doesn't"  # ...或者改用双引号
"doesn't"
>>> '"Yes," they said.'
'"Yes," they said.'
>>> "\"Yes,\" they said."
'"Yes," they said.'
>>> '"Isn\'t," they said.'
'"Isn\'t," they said.'

```

在 Python shell 中，字符串定义和输出字符串看起来可能不同。 print() 函数会略去标示用的引号，并打印经过转义的特殊字符，产生更为易读的输出:>>>

```
>>> s = 'First line.\nSecond line.'  # \n 表示换行符
>>> s  # 不用 print()，特殊字符将包括在字符串中
'First line.\nSecond line.'
```

```
>>> print(s)  # 用 print()，特殊字符会被转写，因此 \n 将产生一个新行
First line.
Second line.

```

如果不希望前置 `\` 的字符转义成特殊字符，可以使用 **原始字符串**，在引号前添加 `r` 即可：

```
>>> print('C:\some\name')  # 这里 \n 表示换行符!
C:\some
ame
>>> print(r'C:\some\name')  # 请注意引号前的 r
C:\some\name

```

原始字符串还有一个微妙的限制：一个原始字符串不能以奇数个 `\` 字符结束；请参阅 此 FAQ 条目 了解更多信息及绕过的办法。

字符串文字可以跨越多行。一种方法是使用三重引号：`"""…"""` 或 `'''…'''` 。行尾会自动包含在字符串中，但可以通过在行尾添加 `\` 来避免这种情况。 在下面的示例中，不包含首行换行符：：

```
>>> print("""\
... Usage: thingy [OPTIONS]
...      -h                        Display this usage message
...      -H hostname               Hostname to connect to
... """)
Usage: thingy [OPTIONS]
     -h                        Display this usage message
     -H hostname               Hostname to connect to

>>>

```

字符串可以用 `+` 合并（粘到一起），也可以用 `*` 重复：

```
>>> # 3 乘以 'un'，再加 'ium'
>>> 3 * 'un' + 'ium'
'unununium'

```

相邻的两个或多个 **字符串字面值** （引号标注的字符）会自动合并：

```
>>> 'Py' 'thon'
'Python'
```

```

```

拼接分隔开的长字符串时，这个功能特别实用：`>>>`

```
>>> text = ('Put several strings within parentheses '
...          'to have them joined together.')
>>> text
'Put several strings within parentheses to have them joined together.'

```

这项功能只能用于两个字面值，不能用于变量或表达式：`>>>`

```
>>> prefix = 'Py'
>>> prefix 'thon'  # 不能拼接变量和字符串字面值
  File "<stdin>", line 1
    prefix 'thon'
           ^^^^^^
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
  File "<stdin>", line 1
    ('un' * 3) 'ium'
               ^^^^^
SyntaxError: invalid syntax

```

合并多个变量，或合并变量与字面值，要用 `+`：`>>>`

```
>>> prefix + 'thon'
'Python'

```

字符串支持 **索引** （下标访问），第一个字符的索引是 0。单字符没有专用的类型，就是长度为一的字符串：`>>>`

```
>>> word = 'Python'
>>> word[0]  # 0 号位的字符
'P'
>>> word[5]  # 5 号位的字符
'n'

```

索引还支持负数，用负数索引时，从右边开始计数：`>>>`

```
>>> word[-1]  # 最后一个字符
'n'
>>> word[-2]  # 倒数第二个字符
'o'
>>> word[-6]
'P'

```

注意，-0 和 0 一样，因此，负数索引从 -1 开始。
除了索引操作，还支持 **切片**。 索引用来获取单个字符，而 **切片** 允许你获取子字符串:>>>

```
>>> word[0:2]  # 从 0 号位 (含) 到 2 号位 (不含) 的字符
'Py'
>>> word[2:5]  # 从 2 号位 (含) 到 5 号位 (不含) 的字符
'tho'

```

切片索引的默认值很有用；省略开始索引时，默认值为 0，省略结束索引时，默认为到字符串的结尾：>>>

```
>>> word[:2]   # 从开头到 2 号位 (不含) 的字符
'Py'
>>> word[4:]   # 从 4 号位 (含) 到末尾
'on'
>>> word[-2:]  # 从倒数第二个 (含) 到末尾
'on'

```

注意，输出结果包含切片开始，但不包含切片结束。因此，`s[:i] + s[i:]` 总是等于 `s`：>>>

```
>>> word[:2] + word[2:]
'Python'
>>> word[:4] + word[4:]
'Python'

```

还可以这样理解切片，索引指向的是字符 **之间** ，第一个字符的左侧标为 0，最后一个字符的右侧标为 **n** ，**n** 是字符串长度。例如：

```
 +---+---+---+---+---+---+
 | P | y | t | h | o | n |
```

```
 +---+---+---+---+---+---+
 0   1   2   3   4   5   6
-6  -5  -4  -3  -2  -1

```

第一行数字是字符串中索引 0...6 的位置，第二行数字是对应的负数索引位置。**i** 到 **j** 的切片由 **i** 和 **j** 之间所有对应的字符组成。
对于使用非负索引的切片，如果两个索引都不越界，切片长度就是起止索引之差。例如， `word[1:3]` 的长度是 2。
索引越界会报错：`>>>`

```
>>> word[42]  # word 只有 6 个字符
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range

```

但是，切片会自动处理越界索引：`>>>`

```
>>> word[4:42]
'on'
>>> word[42:]
''

```

Python 字符串不能修改，是 immutable 的。因此，为字符串中某个索引位置赋值会报错：`>>>`

```
>>> word[0] = 'J'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> word[2:] = 'py'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment

```

要生成不同的字符串，应新建一个字符串：`>>>`

```
>>> 'J' + word[1:]
'Jython'
>>> word[:2] + 'py'
```

```
'Pypy'

```

内置函数 len() 返回字符串的长度：>>>

```
>>> s = 'supercalifragilisticexpialidocious'
>>> len(s)
34

```

参见

文本序列类型 --- str

字符串是 **序列类型** ，支持序列类型的各种操作。

字符串的方法

字符串支持很多变形与查找方法。

f 字符串

内嵌表达式的字符串字面值。

格式字符串语法

使用 str.format() 格式化字符串。

printf 风格的字符串格式化

这里详述了用 `%` 运算符格式化字符串的操作。

3.1.3. 列表

Python 支持多种 **复合** 数据类型，可将不同值组合在一起。最常用的 **列表** ，是用方括号标注，逗号分隔的一组值。**列表** 可以包含不同类型的元素，但一般情况下，各个元素的类型相同：>>>

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]

```

和字符串（及其他内置 sequence 类型）一样，列表也支持索引和切片：>>>

```
>>> squares[0]  # 索引操作将返回条目
1
>>> squares[-1]
25
>>> squares[-3:]  # 切片操作将返回一个新列表
[9, 16, 25]

```

列表还支持合并操作：>>>

```
>>> squares + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

```

与 immutable 字符串不同, 列表是 mutable 类型，其内容可以改变：>>>

```
>>> cubes = [1, 8, 27, 65, 125]  # 这里有点问题
>>> 4 ** 3  # 4 的立方是 64，不是 65!
64
>>> cubes[3] = 64  # 替换错误的值
>>> cubes
[1, 8, 27, 64, 125]

```

你也可以在通过使用 list.append() **方法**，在列表末尾添加新条目（我们将在后面介绍更多相关的方法）:>>>

```
>>> cubes.append(216)  # 添加 6 的立方
>>> cubes.append(7 ** 3)  # 和 7 的立方
>>> cubes
[1, 8, 27, 64, 125, 216, 343]

```

Python 中的简单赋值绝不会复制数据。 当你将一个列表赋值给一个变量时，该变量将引用 **现有的列表**。你通过一个变量对列表所做的任何更改都会被引用它的所有其他变量看到。:>>>

```
>>> rgb = ["Red", "Green", "Blue"]
>>> rgba = rgb
>>> id(rgb) == id(rgba)  # 它们指向同一个对象
True
>>> rgba.append("Alph")
>>> rgb
["Red", "Green", "Blue", "Alph"]

```

切片操作返回包含请求元素的新列表。以下切片操作会返回列表的 浅拷贝：>>>

```
>>> correct_rgba = rgba[:]
>>> correct_rgba[-1] = "Alpha"
>>> correct_rgba
```

```
["Red", "Green", "Blue", "Alpha"]
>>> rgba
["Red", "Green", "Blue", "Alph"]

```

为切片赋值可以改变列表大小，甚至清空整个列表：`>>>`

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> # 替换一些值
>>> letters[2:5] = ['C', 'D', 'E']
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> # 现在移除它们
>>> letters[2:5] = []
>>> letters
['a', 'b', 'f', 'g']
>>> # 通过用一个空列表替代所有元素来清空列表
>>> letters[:] = []
>>> letters
[]

```

内置函数 len() 也支持列表：`>>>`

```
>>> letters = ['a', 'b', 'c', 'd']
>>> len(letters)
4

```

还可以嵌套列表（创建包含其他列表的列表），例如：`>>>`

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

```
10
```

3.2. 走向编程的第一步

当然，我们还能用 Python 完成比二加二更复杂的任务。 例如，我们可以像下面这样写出 斐波那契数列 初始部分的子序列:>>>

```
>>> # 斐波那契数列：
>>> # 前两项之和即下一项的值
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a+b
...
0
1
1
2
3
5
8

```

本例引入了几个新功能。

- 第一行中的

 多重赋值 ： 变量 a 和 b 同时获得新值 0 和 1 。最后一行又用了一次多重赋值，体现了，等号右边的所有表达式的值，都是在这一语句对任何变量赋新值之前求出来的——求值顺序为从左到右。

- while

 循环只要条件（这里是 a < 10）为真就会一直执行。Python 和 C 一样，任何非零整数都为真，零为假。这个条件也可以是字符串或列表类型的值，事实上，任何序列都可以：长度非零就为真，空序列则为假。示例中的判断只是最简单的比较。比较操作符的写法和 C 语言一样： < （小于）、 > （大于）、 == （等于）、 <= （小于等于)、 >= （大于等于）及 != （不等于）。

- 循环体

 是 **缩进的** ： 缩进是 Python 组织语句的方式。在交互式命令行里，得为每个缩进的行输入空格（或制表符）。使用文本编辑器可以实现更复杂的输入方式；所有像样的文本编辑器都支持自动缩进。交互式输入复合语句时，要在最后输入空白行表示完成（因为解析器不知道哪一行代码是代码块的最后一行）。注意，同一块语句的每一行的缩进相同。

- print()

函数输出给定参数的值。 除了可以以单一的表达式作为参数（比如，前面的计算器的例子），它还能处理多个参数，包括浮点数与字符串。 它输出的字符串不带引号，且各参数项之间会插入一个空格，这样可以实现更好的格式化操作，就像这样:>>>

```
>>> i = 256*256
>>> print('The value of i is', i)
The value of i is 65536

```

关键字参数 **end** 可以取消输出后面的换行, 或用另一个字符串结尾：>>>

```
>>> a, b = 0, 1
>>> while a < 1000:
...     print(a, end=',')
...     a, b = b, a+b
...
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

4. 更多控制流工具

除了刚介绍的 while 语句，Python 还用了一些别的。我们将在本章中遇到它们。

4.1. if 语句

最让人耳熟能详的语句应当是 if 语句：>>>

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
更多

```

可有零个或多个 elif 部分，else 部分也是可选的。关键字 'elif' 是 'else if' 的缩写，用于避免过多的缩进。if ... elif ... elif ... 序列可以当作其它语言中 switch 或 case 语句的替代品。
如果是把一个值与多个常量进行比较，或者检查特定类型或属性，match 语句更有用。详见 match 语句。

4.2. for 语句

Python 的 for 语句与 C 或 Pascal 中的不同。Python 的 for 语句不迭代算术递增数值（如 Pascal），或是给予用户定义迭代步骤和结束条件的能力（如 C），而是在列表或字符串等任意序列的元素上迭代，按它们在序列中出现的顺序。 例如（这不是有意要暗指什么）： >>>

```
>>> # 度量一些字符串:
>>> words = ['cat', 'window', 'defenestrate']
>>> for w in words:
...     print(w, len(w))
...
cat 3
window 6
defenestrate 12

```

很难正确地在迭代多项集的同时修改多项集的内容。更简单的方法是迭代多项集的副本或者创建新的多项集：

```
# 创建示例多项集
users = {'Hans': 'active', 'Éléonore': 'inactive', '景太郎': 'active'}

# 策略：迭代一个副本
for user, status in users.copy().items():
    if status == 'inactive':
        del users[user]

# 策略：创建一个新多项集
active_users = {}
for user, status in users.items():
    if status == 'active':
        active_users[user] = status

```

4.3. range() 函数

内置函数 range() 用于生成等差数列：>>>

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4

```

生成的序列绝不会包括给定的终止值；range(10) 生成 10 个值——长度为 10 的序列的所有合法索引。range 可以不从 0 开始，且可以按给定的步长递增（即使是负数步长）：>>>

```
>>> list(range(5, 10))
[5, 6, 7, 8, 9]

>>> list(range(0, 10, 3))
[0, 3, 6, 9]

>>> list(range(-10, -100, -30))
[-10, -40, -70]

```

要按索引迭代序列，可以组合使用 range() 和 len()：>>>

```
>>> a = ['Mary', 'had', 'a', 'little', 'lamb']
>>> for i in range(len(a)):
...     print(i, a[i])
...
0 Mary
1 had
2 a
3 little
4 lamb

```

不过大多数情况下 enumerate() 函数很方便，详见 循环的技巧。

如果直接打印一个 range 会发生意想不到的事情：>>>

```
>>> range(10)
range(0, 10)

```

range() 返回的对象在很多方面和列表的行为一样，但其实它和列表不一样。该对象只有在被迭代时才一个一个地返回所期望的列表项，并没有真正生成过一个含有全部项的列表，从而节省了空间。
这种对象称为可迭代对象 iterable，适合作为需要获取一系列值的函数或程序构件的参数。for 语句就是这样的程序构件；以可迭代对象作为参数的函数例如 sum()：>>>

```
>>> sum(range(4))  # 0 + 1 + 2 + 3
6

```

之后我们会看到更多返回可迭代对象，或以可迭代对象作为参数的函数。在 数据结构 这一章中，我们将讨论 list() 的更多细节。

4.4. break 和 continue 语句

break 语句将跳出最近的一层 for 或 while 循环:>>>

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(f"{n} equals {x} * {n//x}")
...             break
...
4 equals 2 * 2
6 equals 2 * 3
8 equals 2 * 4
9 equals 3 * 3

```

continue 语句将继续执行循环的下一次迭代:>>>

```
>>> for num in range(2, 10):
...     if num % 2 == 0:
...         print(f"Found an even number {num}")
...         continue
...     print(f"Found an odd number {num}")
...
```

```
Found an even number 2
Found an odd number 3
Found an even number 4
Found an odd number 5
Found an even number 6
Found an odd number 7
Found an even number 8
Found an odd number 9

```

4.5. 循环的 else 子句

在 for 或 while 循环中 break 语句可能对应一个 else 子句。 如果循环在未执行 break 的情况下结束，else 子句将会执行。

在 for 循环中，else 子句会在循环结束其他最后一次迭代之后，即未执行 break 的情况下被执行。

在 while 循环中，它会在循环条件变为假值后执行。

在这两类循环中，当在循环被 break 终结时 else 子句 **不会** 被执行。 当然，其他提前结束循环的方式，如 return 或是引发异常，也会跳过 else 子句的执行。

下面的搜索质数的 for 循环就是一个例子： >>>

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(n, 'equals', x, '*', n//x)
...             break
...     else:
...         # 循环到底未找到一个因数
...         print(n, 'is a prime number')
...
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3

```

（对，这是正确的代码。 仔细看：其中 else 子句属于 for 循环，而 **不属于** if 语句。）
分析 else 子句的一种方式是想象它对应于循环内的 if。 当循环执行时，它将运行一系列的 if/if/if/else。 if 位于循环内部，会出现多次。 当出现条件为真的情况时，将发生 break。 如果条件一直不为真，则循环外的 else 子句将被执行。
当配合循环使用时，else 子句更像是 try 语句的 else 子句而不像 if 语句的相应子句：一个 try 语句的 else 子句会在未发生异常时运行，而一个循环的 else 子句会在未发生 break 时运行。 有关 try 语句和异常的详情，请参阅 异常的处理。

4.6. pass 语句

pass 语句不执行任何动作。语法上需要一个语句，但程序毋需执行任何动作时，可以使用该语句。例如：>>>

```
>>> while True:
...     pass  # 无限等待键盘中断 (Ctrl+C)
...

```

这常用于创建一个最小的类：>>>

```
>>> class MyEmptyClass:
...     pass
...

```

pass 还可用作函数或条件语句体的占位符，让你保持在更抽象的层次进行思考。pass 会被默默地忽略：>>>

```
>>> def initlog(*args):
...     pass   # 记得实现这个!
...

```

4.7. match 语句

match 语句接受一个表达式并把它的值与一个或多个 case 块给出的一系列模式进行比较。这表面上像 C、Java 或 JavaScript（以及许多其他程序设计语言）中的 switch 语句，但其实它更像 Rust 或 Haskell 中的模式匹配。只有第一个匹配的模式会被执行，并且它还可以提取值的组成部分（序列的元素或对象的属性）赋给变量。
最简单的形式是将一个主语值与一个或多个字面值进行比较：

```python
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
        case _:
            return "Something's wrong with the internet"

```

注意最后一个代码块：“变量名” `_` 被作为 **通配符** 并必定会匹配成功。如果没有 case 匹配成功，则不会执行任何分支。

你可以用 `|` （“或”）将多个字面值组合到一个模式中：

```python
case 401 | 403 | 404:
    return "Not allowed"

```

形如解包赋值的模式可被用于绑定变量：

```python
# point 是一个 (x, y) 元组
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Not a point")

```

请仔细学习此代码！第一个模式有两个字面值，可视为前述字面值模式的扩展。接下来的两个模式结合了一个字面值和一个变量，变量 **绑定** 了来自主语（`point`）的一个值。第四个模式捕获了两个值，使其在概念上与解包赋值 `(x, y) = point` 类似。

如果用类组织数据，可以用“类名后接一个参数列表”这种很像构造器的形式，把属性捕获到变量里：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def where_is(point):
    match point:
        case Point(x=0, y=0):
            print("Origin")
        case Point(x=0, y=y):
            print(f"Y={y}")
        case Point(x=x, y=0):
            print(f"X={x}")
        case Point():
            print("Somewhere else")
        case _:
            print("Not a point")

```

一些内置类（如 dataclass）为属性提供了一个顺序，此时，可以使用位置参数。自定义类可通过在类中设置特殊属性 __match_args__，为属性指定其在模式中对应的位置。若设为 ("x", "y")，则以下模式相互等价（且都把属性 y 绑定到变量 var）：

```
Point(1, var)
Point(1, y=var)
Point(x=1, y=var)
Point(y=var, x=1)

```

建议这样来阅读一个模式——通过将其视为赋值语句等号左边的一种扩展形式，来理解各个变量被设为何值。match 语句只会为单一的名称（如上面的 var）赋值，而不会赋值给带点号的名称（如 foo.bar）、属性名（如上面的 x= 和 y=）和类名（是通过其后的 "(...)" 来识别的，如上面的 Point）。

模式可以任意嵌套。举例来说，如果我们有一个由 Point 组成的列表，且 Point 添加了 __match_args__ 时，我们可以这样来匹配它：

```
class Point:
```

```
    __match_args__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

match points:
    case []:
        print("No points")
    case [Point(0, 0)]:
        print("The origin")
    case [Point(x, y)]:
        print(f"Single point {x}, {y}")
    case [Point(0, y1), Point(0, y2)]:
        print(f"Two on the Y axis at {y1}, {y2}")
    case _:
        print("Something else")

```

我们可以为模式添加 `if` 作为守卫子句。如果守卫子句的值为假，那么 `match` 会继续尝试匹配下一个 case 块。注意是先将值捕获，再对守卫子句求值：

```
match point:
    case Point(x, y) if x == y:
        print(f"Y=X at {x}")
    case Point(x, y):
        print(f"Not on the diagonal")

```

该语句的一些其它关键特性：

- 与解包赋值类似，元组和列表模式具有完全相同的含义并且实际上都能匹配任意序列，区别是它们不能匹配迭代器或字符串。

- 序列模式支持扩展解包：

`[x, y, *rest]` 和 `(x, y, *rest)` 和相应的解包赋值做的事是一样的。接在 `*` 后的名称也可以为 `_`，所以 `(x, y, *_)` 匹配含至少两项的序列，而不必绑定剩余的项。

- 映射模式：

`{"bandwidth": b, "latency": l}` 从字典中捕获 `"bandwidth"` 和 `"latency"` 的值。额外的键会被忽略，这一点与序列模式不同。`**rest` 这样的解包也支持。（但 `**_` 将会是冗余的，故不允许使用。）

- 使用

`as` 关键字可以捕获子模式：

```
case (Point(x1, y1), Point(x2, y2) as p2): ...

```

将把输入中的第二个元素捕获为 p2 （只要输入是包含两个点的序列）

- 大多数字面值是按相等性比较的，但是单例对象

True、False 和 None 则是按 id 比较的。

- 模式可以使用具名常量。它们必须作为带点号的名称出现，以防止它们被解释为用于捕获的变量：

```
from enum import Enum
class Color(Enum):
    RED = 'red'
    GREEN = 'green'
    BLUE = 'blue'

color = Color(input("Enter your choice of 'red', 'blue' or 'green': "))

match color:
    case Color.RED:
        print("I see red!")
    case Color.GREEN:
        print("Grass is green")
    case Color.BLUE:
        print("I'm feeling the blues :(")

```

更详细的说明和更多示例，可参阅以教程格式撰写的 **PEP 636**。

4.8. 定义函数

下列代码创建一个可以输出限定数值内的斐波那契数列函数： `>>>`

```
>>> def fib(n):    # 打印小于 n 的斐波那契数列
...     """Print a Fibonacci series less than n."""
...     a, b = 0, 1
...     while a < n:
...         print(a, end=' ')
...         a, b = b, a+b
...     print()
...
```

```
>>> # 现在调用我们刚定义的函数：
>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597

```

定义 函数使用关键字 def，后跟函数名与括号内的形参列表。函数语句从下一行开始，并且必须缩进。

函数内的第一条语句是字符串时，该字符串就是文档字符串，也称为 **docstring**，详见 文档字符串。利用文档字符串可以自动生成在线文档或打印版文档，还可以让开发者在浏览代码时直接查阅文档；Python 开发者最好养成在代码中加入文档字符串的好习惯。

函数在 **执行** 时使用函数局部变量符号表，所有函数变量赋值都存在局部符号表中；引用变量时，首先，在局部符号表里查找变量，然后，是外层函数局部符号表，再是全局符号表，最后是内置名称符号表。因此，尽管可以引用全局变量和外层函数的变量，但最好不要在函数内直接赋值（除非是 global 语句定义的全局变量，或 nonlocal 语句定义的外层函数变量）。

在调用函数时会将实际参数（实参）引入到被调用函数的局部符号表中；因此，实参是使用 **按值调用** 来传递的（其中的 **值** 始终是对象的 **引用** 而不是对象的值）。 [1] 当一个函数调用另外一个函数时，会为该调用创建一个新的局部符号表。

函数定义在当前符号表中把函数名与函数对象关联在一起。解释器把函数名指向的对象作为用户自定义函数。还可以使用其他名称指向同一个函数对象，并访问访该函数：>>>

```
>>> fib
<function fib at 10042ed0>
>>> f = fib
>>> f(100)
0 1 1 2 3 5 8 13 21 34 55 89

```

如果你用过其他语言，你可能会认为 fib 不是函数而是一个过程，因为它没有返回值。 事实上，即使没有 return 语句的函数也有返回值，尽管这个值可能相当无聊。 这个值被称为 None (是一个内置名称)。 通常解释器会屏蔽单独的返回值 None。 如果你确有需要可以使用 print() 查看它:>>>

```
>>> fib(0)
>>> print(fib(0))
None

```

编写不直接输出斐波那契数列运算结果，而是返回运算结果列表的函数也非常简单：>>>

```
>>> def fib2(n):  # 返回斐波那契数组直到 n
...     """Return a list containing the Fibonacci series up to n."""
...     result = []
```

```
...     a, b = 0, 1
...     while a < n:
...         result.append(a)    # 见下
...         a, b = b, a+b
...     return result
...
>>> f100 = fib2(100)    # 调用它
>>> f100                # 输出结果
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

本例也新引入了一些 Python 功能：

- return

语句返回函数的值。return 语句不带表达式参数时，返回 None。函数执行完毕退出也返回 None。

- 语句

result.append(a) 调用了列表对象 result 的 **方法**。 方法是‘从属于’对象的函数，其名称为 obj.methodname，其中 obj 是某个对象（可以是一个表达式），methodname 是由对象的类型定义的方法名称。 不同的类型定义了不同的方法。 不同的类型的方法可以使用相同的名称而不会产生歧义。 （使用 **类** 可以定义自己的对象类型和方法，参见 类。） 在示例中显示的方法 append() 是由列表对象定义的；它会在列表的末尾添加一个新元素。 在本例中它等同于 result = result + [a]，但效率更高。

4.9. 函数定义详解

函数定义支持可变数量的参数。这里列出三种可以组合使用的形式。

4.9.1. 默认值参数

为参数指定默认值是非常有用的方式。调用函数时，可以使用比定义时更少的参数，例如：

```
def ask_ok(prompt, retries=4, reminder='Please try again!'):
    while True:
        reply = input(prompt)
        if reply in {'y', 'ye', 'yes'}:
            return True
        if reply in {'n', 'no', 'nop', 'nope'}:
            return False
        retries = retries - 1
        if retries < 0:
            raise ValueError('invalid user response')
        print(reminder)
```

该函数可以用以下方式调用：

- 只给出必选实参：

`ask_ok('Do you really want to quit?')`

- 给出一个可选实参：

`ask_ok('OK to overwrite the file?', 2)`

- 给出所有实参：

`ask_ok('OK to overwrite the file?', 2, 'Come on, only yes or no!')`

本例还使用了关键字 in ，用于确认序列中是否包含某个值。

默认值在 **定义** 作用域里的函数定义中求值，所以：

```
i = 5

def f(arg=i):
    print(arg)

i = 6
f()

```

上例输出的是 `5`。

重要警告： 默认值只计算一次。默认值为列表、字典或类实例等可变对象时，会产生与该规则不同的结果。例如，下面的函数会累积后续调用时传递的参数：

```
def f(a, L=[]):
    L.append(a)
    return L

print(f(1))
print(f(2))
print(f(3))

```

输出结果如下：

```
[1]
[1, 2]
[1, 2, 3]

```

不想在后续调用之间共享默认值时，应以如下方式编写函数：

```
def f(a, L=None):
    if L is None:
        L = []
    L.append(a)
    return L

```

4.9.2. 关键字参数

kwarg=value 形式的 关键字参数 也可以用于调用函数。函数示例如下：

```
def parrot(voltage, state='a stiff', action='voom', type='Norwegian Blue'):
    print("-- This parrot wouldn't", action, end=' ')
    print("if you put", voltage, "volts through it.")
    print("-- Lovely plumage, the", type)
    print("-- It's", state, "!")

```

该函数接受一个必选参数（voltage）和三个可选参数（state, action 和 type）。该函数可用下列方式调用：

```
parrot(1000)                                          # 1 个位置参数
parrot(voltage=1000)                                  # 1 个关键字参数
parrot(voltage=1000000, action='VOOOOOM')             # 2 个关键字参数
parrot(action='VOOOOOM', voltage=1000000)             # 2 个关键字参数
parrot('a million', 'bereft of life', 'jump')         # 3 个位置参数
parrot('a thousand', state='pushing up the daisies')  # 1 个位置参数，1 个关键字参数

```

以下调用函数的方式都无效：

```
parrot()                     # 缺失必需的参数
parrot(voltage=5.0, 'dead')  # 关键字参数后存在非关键字参数
parrot(110, voltage=220)     # 同一个参数重复的值
parrot(actor='John Cleese')  # 未知的关键字参数

```

函数调用时，关键字参数必须跟在位置参数后面。所有传递的关键字参数都必须匹配一个函数接受的参数（比如，actor 不是函数 parrot 的有效参数），关键字参数的顺序并不重要。这也包括必选参

数，（比如，parrot(voltage=1000) 也有效）。不能对同一个参数多次赋值，下面就是一个因此限制而失败的例子：>>>

```
>>> def function(a):
...     pass
...
>>> function(0, a=0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: function() got multiple values for argument 'a'

```

最后一个形参为 **name 形式时，接收一个字典（详见 映射类型 --- dict），该字典包含与函数中已定义形参对应之外的所有关键字参数。**name 形参可以与 *name 形参（下一小节介绍）组合使用（*name 必须在 **name 前面），*name 形参接收一个 元组，该元组包含形参列表之外的位置参数。例如，可以定义下面这样的函数：

```
def cheeseshop(kind, *arguments, **keywords):
    print("-- Do you have any", kind, "?")
    print("-- I'm sorry, we're all out of", kind)
    for arg in arguments:
        print(arg)
    print("-" * 40)
    for kw in keywords:
        print(kw, ":", keywords[kw])

```

该函数可以用如下方式调用：

```
cheeseshop("Limburger", "It's very runny, sir.",
           "It's really very, VERY runny, sir.",
           shopkeeper="Michael Palin",
           client="John Cleese",
           sketch="Cheese Shop Sketch")

```

输出结果如下：

```
-- Do you have any Limburger ?
-- I'm sorry, we're all out of Limburger
It's very runny, sir.
```

```
It's really very, VERY runny, sir.
----------------------------------------
shopkeeper : Michael Palin
client : John Cleese
sketch : Cheese Shop Sketch

```

注意，关键字参数在输出结果中的顺序与调用函数时的顺序一致。

4.9.3. 特殊参数

默认情况下，参数可以按位置或显式关键字传递给 Python 函数。为了让代码易读、高效，最好限制参数的传递方式，这样，开发者只需查看函数定义，即可确定参数项是仅按位置、按位置或关键字，还是仅按关键字传递。
函数定义如下：

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
      -----------    ----------     ----------
        |             |                  |
        |        位置或关键字    |
        |                                  - 仅限关键字
         -- 仅限位置

```

/ 和 * 是可选的。这些符号表明形参如何把参数值传递给函数：位置、位置或关键字、关键字。关键字形参也叫作命名形参。

4.9.3.1. 位置或关键字参数

函数定义中未使用 / 和 * 时，参数可以按位置或关键字传递给函数。

4.9.3.2. 仅位置参数

此处再介绍一些细节，特定形参可以标记为 **仅限位置**。**仅限位置** 时，形参的顺序很重要，且这些形参不能用关键字传递。仅限位置形参应放在 / （正斜杠）前。/ 用于在逻辑上分割仅限位置形参与其它形参。如果函数定义中没有 /，则表示没有仅限位置形参。
/ 后可以是 **位置或关键字** 或 **仅限关键字** 形参。

4.9.3.3. 仅限关键字参数

把形参标记为 **仅限关键字**，表明必须以关键字参数形式传递该形参，应在参数列表中第一个 **仅限关键字** 形参前添加 *。

4.9.3.4. 函数示例

请看下面的函数定义示例，注意 `/` 和 `*` 标记：`>>>`

```
>>> def standard_arg(arg):
...     print(arg)
...
>>> def pos_only_arg(arg, /):
...     print(arg)
...
>>> def kwd_only_arg(*, arg):
...     print(arg)
...
>>> def combined_example(pos_only, /, standard, *, kwd_only):
...     print(pos_only, standard, kwd_only)

```

第一个函数定义 `standard_arg` 是最常见的形式，对调用方式没有任何限制，可以按位置也可以按关键字传递参数：`>>>`

```
>>> standard_arg(2)
2

>>> standard_arg(arg=2)
2

```

第二个函数 `pos_only_arg` 的函数定义中有 `/`，仅限使用位置形参：`>>>`

```
>>> pos_only_arg(1)
1

>>> pos_only_arg(arg=1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: pos_only_arg() got some positional-only arguments passed as keyword arguments: 'arg'

```

第三个函数 `kwd_only_arg` 如在函数定义中通过 `*` 所指明的那样只允许关键字参数。`>>>`

```
>>> kwd_only_arg(3)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: kwd_only_arg() takes 0 positional arguments but 1 was given

>>> kwd_only_arg(arg=3)
3

```

最后一个函数在同一个函数定义中，使用了全部三种调用惯例：`>>>`

```
>>> combined_example(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: combined_example() takes 2 positional arguments but 3 were given

>>> combined_example(1, 2, kwd_only=3)
1 2 3

>>> combined_example(1, standard=2, kwd_only=3)
1 2 3

>>> combined_example(pos_only=1, standard=2, kwd_only=3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: combined_example() got some positional-only arguments passed as keyword arguments: 'pos_only'

```

下面的函数定义中，`kwds` 把 `name` 当作键，因此，可能与位置参数 `name` 产生潜在冲突：

```
def foo(name, **kwds):
    return 'name' in kwds

```

调用该函数不可能返回 `True`，因为关键字 `'name'` 总与第一个形参绑定。例如：`>>>`

```
>>> foo(1, **{'name': 2})
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: foo() got multiple values for argument 'name'
>>>
```

```

```

加上 `/` （仅限位置参数）后，就可以了。此时，函数定义把 `name` 当作位置参数，`'name'` 也可以作为关键字参数的键：`>>>`

```
>>> def foo(name, /, **kwds):
...     return 'name' in kwds
...
>>> foo(1, **{'name': 2})
True

```

换句话说，仅限位置形参的名称可以在 `**kwds` 中使用，而不产生歧义。

4.9.3.5. 小结

以下用例决定哪些形参可以用于函数定义：

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):

```

说明：

- 使用仅限位置形参，可以让用户无法使用形参名。形参名没有实际意义时，强制调用函数的实参顺序时，或同时接收位置形参和关键字时，这种方式很有用。

- 当形参名有实际意义，且显式名称可以让函数定义更易理解时，阻止用户依赖传递实参的位置时，才使用关键字。

- 对于 API，使用仅限位置形参，可以防止未来修改形参名时造成破坏性的 API 变动。

4.9.4. 任意实参列表

调用函数时，使用任意数量的实参是最少见的选项。这些实参包含在元组中（详见 元组和序列 ）。在可变数量的实参之前，可能有若干个普通参数：

```
def write_multiple_items(file, separator, *args):
    file.write(separator.join(args))

```

variadic 参数用于采集传递给函数的所有剩余参数，因此，它们通常在形参列表的末尾。`*args` 形参后的任何形式参数只能是仅限关键字参数，即只能用作关键字参数，不能用作位置参数：`>>>`

```
>>> def concat(*args, sep="/"):
...     return sep.join(args)
...
>>> concat("earth", "mars", "venus")
'earth/mars/venus'
>>> concat("earth", "mars", "venus", sep=".")
'earth.mars.venus'

```

4.9.5. 解包实参列表

函数调用要求独立的位置参数，但实参在列表或元组里时，要执行相反的操作。例如，内置的 range() 函数要求独立的 **start** 和 **stop** 实参。如果这些参数不是独立的，则要在调用函数时，用 `*` 操作符把实参从列表或元组解包出来：`>>>`

```
>>> list(range(3, 6))            # 附带两个参数的正常调用
[3, 4, 5]
>>> args = [3, 6]
>>> list(range(*args))            # 附带从一个列表解包的参数的调用
[3, 4, 5]

```

同样，字典可以用 `**` 操作符传递关键字参数：`>>>`

```
>>> def parrot(voltage, state='a stiff', action='voom'):
...     print("-- This parrot wouldn't", action, end=' ')
...     print("if you put", voltage, "volts through it.", end=' ')
...     print("E's", state, "!")
...
>>> d = {"voltage": "four million", "state": "bleedin' demised", "action": "VOOM"}
>>> parrot(**d)
-- This parrot wouldn't VOOM if you put four million volts through it. E's bleedin' demised !

```

4.9.6. Lambda 表达式

lambda 关键字用于创建小巧的匿名函数。`lambda a, b: a+b` 函数返回两个参数的和。Lambda 函数可用于任何需要函数对象的地方。在语法上，匿名函数只能是单个表达式。在语义上，它只是常规函数定义的语法糖。与嵌套函数定义一样，lambda 函数可以引用包含作用域中的变量：

```
>>> def make_incrementor(n):
...     return lambda x: x + n
...
>>> f = make_incrementor(42)
>>> f(0)
42
>>> f(1)
43

```

上例用 lambda 表达式返回函数。还可以把匿名函数用作传递的实参：

```
>>> pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
>>> pairs.sort(key=lambda pair: pair[1])
>>> pairs
[(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]

```

4.9.7. 文档字符串

以下是文档字符串内容和格式的约定。

第一行应为对象用途的简短摘要。为保持简洁，不要在这里显式说明对象名或类型，因为可通过其他方式获取这些信息（除非该名称碰巧是描述函数操作的动词）。这一行应以大写字母开头，以句点结尾。

文档字符串为多行时，第二行应为空白行，在视觉上将摘要与其余描述分开。后面的行可包含若干段落，描述对象的调用约定、副作用等。

Python 解析器不会删除 Python 中多行字符串字面值的缩进，因此，文档处理工具应在必要时删除缩进。这项操作遵循以下约定：文档字符串第一行 **之后** 的第一个非空行决定了整个文档字符串的缩进量（第一行通常与字符串开头的引号相邻，其缩进在字符串中并不明显，因此，不能用第一行的缩进），然后，删除字符串中所有行开头处与此缩进“等价”的空白符。不能有比此缩进更少的行，但如果出现了缩进更少的行，应删除这些行的所有前导空白符。转化制表符后（通常为 8 个空格），应测试空白符的等效性。

下面是多行文档字符串的一个例子：

```
>>> def my_function():
```

```
...     """Do nothing, but document it.
...
...     No, really, it doesn't do anything.
...     """
...     pass
...
>>> print(my_function.__doc__)
Do nothing, but document it.

    No, really, it doesn't do anything.

```

4.9.8. 函数注解

函数注解 是可选的用户自定义函数类型的元数据完整信息（详见 **PEP 3107** 和 **PEP 484** ）。

标注 以字典的形式存放在函数的 __annotations__ 属性中而对函数的其他部分没有影响。 形参标注的定义方式是在形参名后加冒号，后面跟一个会被求值为标注的值的表达式。 返回值标注的定义方式是加组合符号 `->`，后面跟一个表达式，这样的校注位于形参列表和表示 def 语句结束的冒号。 下面的示例有一个必须的参数、一个可选的关键字参数以及返回值都带有相应的标注:>>>

```
>>> def f(ham: str, eggs: str = 'eggs') -> str:
...     print("Annotations:", f.__annotations__)
...     print("Arguments:", ham, eggs)
...     return ham + ' and ' + eggs
...
>>> f('spam')
Annotations: {'ham': <class 'str'>, 'return': <class 'str'>, 'eggs': <class 'str'>}
Arguments: spam eggs
'spam and eggs'

```

4.10. 小插曲：编码风格

现在你将要写更长，更复杂的 Python 代码，是时候讨论一下 **代码风格** 了。 大多数语言都能以不同的风格被编写（或更准确地说，被格式化）；有些比其他的更具有可读性。 能让其他人轻松阅读你的代码总是一个好主意，采用一种好的编码风格对此有很大帮助。

Python 项目大多都遵循 **PEP 8** 的风格指南；它推行的编码风格易于阅读、赏心悦目。Python 开发者均应抽时间悉心研读；以下是该提案中的核心要点：

- 缩进，用 4 个空格，不要用制表符。

4 个空格是小缩进（更深嵌套）和大缩进（更易阅读）之间的折中方案。制表符会引起混乱，最好别用。

- 换行，一行不超过 79 个字符。

这样换行的小屏阅读体验更好，还便于在大屏显示器上并排阅读多个代码文件。

- 用空行分隔函数和类，及函数内较大的代码块。

- 最好把注释放到单独一行。

- 使用文档字符串。

- 运算符前后、逗号后要用空格，但不要直接在括号内使用：

`a = f(1, 2) + g(3, 4)`。

- 类和函数的命名要一致；按惯例，命名类用

`UpperCamelCase`，命名函数与方法用 `lowercase_with_underscores`。命名方法中第一个参数总是用 `self` (类和方法详见 初探类)。

- 编写用于国际多语环境的代码时，不要用生僻的编码。Python 默认的 UTF-8 或纯 ASCII 可以胜任各种情况。

- 同理，就算多语阅读、维护代码的可能再小，也不要在标识符中使用非 ASCII 字符。

5. 数据结构

本章深入讲解之前学过的一些内容，同时，还增加了新的知识点。

5.1. 列表详解

列表数据类型支持很多方法，列表对象的所有方法所示如下：

list.**append**(**x**)

在列表末尾添加一项。 类似于 `a[len(a):] = [x]`。

list.**extend**(**iterable**)

通过添加来自 iterable 的所有项来扩展列表。 类似于 `a[len(a):] = iterable`。

list.**insert**(**i**, **x**)

在指定位置插入元素。第一个参数是插入元素的索引，因此，`a.insert(0, x)` 在列表开头插入元素， `a.insert(len(a), x)` 等同于 `a.append(x)` 。

list.**remove**(**x**)

从列表中删除第一个值为 **x** 的元素。未找到指定元素时，触发 ValueError 异常。

list.**pop**([**i**])

移除列表中给定位置上的条目，并返回该条目。 如果未指定索引号，则 `a.pop()` 将移除并返回列表中的最后一个条目。 如果列表为空或索引号在列表索引范围之外则会引发 IndexError。

list.**clear**()

移除列表中的所有项。 类似于 `del a[:]`。

list.**index**(**x**[, **start**[, **end**]])

返回列表中第一个值为 **x** 的元素的零基索引。未找到指定元素时，触发 ValueError 异常。可选参数 **start** 和 **end** 是切片符号，用于将搜索限制为列表的特定子序列。返回的索引是相对于整个序列的开始计算的，而不是 **start** 参数。

list.**count**(**x**)

返回列表中元素 **x** 出现的次数。

list.**sort**(*, **key=None**, **reverse=False**)

就地排序列表中的元素（要了解自定义排序参数，详见 sorted()）。

list.**reverse**()

翻转列表中的元素。

list.**copy**()

返回列表的浅拷贝。 类似于 `a[:]`。

多数列表方法示例： `>>>`

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.count('tangerine')
0
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)  # 从 4 号位开始查找下一个 banana
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits
```

```
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange', 'grape']
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'

```

你可能已经注意到 insert, remove 或 sort 等仅修改列表的方法都不会打印返回值 -- 它们返回默认值 None。 [1] 这是适用于 Python 中所有可变数据结构的设计原则。
你可能会注意到的另一件事是并非所有数据都可以排序或比较。 举例来说，[None, 'hello', 10] 就不可排序因为整数不能与字符串比较而 None 不能与其他类型比较。 此外，还存在一些没有定义顺序关系的类型。 例如，3+4j < 5+7j 就不是一个合法的比较。

5.1.1. 用列表实现堆栈

列表方法使得将列表用作栈非常容易，最后添加的元素会最先被取出（“后进先出”）。 要将一个条目添加到栈顶，可使用 append()。 要从栈顶取出一个条目，则使用 pop() 且不必显式指定索引。 例如:>>>

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]

```

5.1.2. 用列表实现队列

列表也可以用作队列，最先加入的元素，最先取出（“先进先出”）；然而，列表作为队列的效率很低。因为，在列表末尾添加和删除元素非常快，但在列表开头插入或移除元素却很慢（因为所有其他元素都必须移动一位）。

实现队列最好用 collections.deque，可以快速从两端添加或删除元素。例如：

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # Terry 到了
>>> queue.append("Graham")          # Graham 到了
>>> queue.popleft()                 # 第一个到的现在走了
'Eric'
>>> queue.popleft()                 # 第二个到的现在走了
'John'
>>> queue                           # 按到达顺序排列的剩余队列
deque(['Michael', 'Terry', 'Graham'])

```

5.1.3. 列表推导式

列表推导式创建列表的方式更简洁。常见的用法为，对序列或可迭代对象中的每个元素应用某种操作，用生成的结果创建新的列表；或用满足特定条件的元素创建子序列。

例如，创建平方值的列表：

```
>>> squares = []
>>> for x in range(10):
...     squares.append(x**2)
...
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```

注意，这段代码创建（或覆盖）变量 `x`，该变量在循环结束后仍然存在。下述方法可以无副作用地计算平方列表：

```
squares = list(map(lambda x: x**2, range(10)))

```

或等价于：

```
squares = [x**2 for x in range(10)]
```

上面这种写法更简洁、易读。
列表推导式的方括号内包含以下内容：一个表达式，后面为一个 for 子句，然后，是零个或多个 for 或 if 子句。结果是由表达式依据 for 和 if 子句求值计算而得出一个新列表。 举例来说，以下列表推导式将两个列表中不相等的元素组合起来：

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

等价于：

```
>>> combs = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
...             combs.append((x, y))
...
>>> combs
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

注意，上面两段代码中，for 和 if 的顺序相同。
表达式是元组（例如上例的 (x, y)）时，必须加上括号：

```
>>> vec = [-4, -2, 0, 2, 4]
>>> # 新建一个将值翻倍的列表
>>> [x*2 for x in vec]
[-8, -4, 0, 4, 8]
>>> # 过滤列表以排除负数
>>> [x for x in vec if x >= 0]
[0, 2, 4]
>>> # 对所有元素应用一个函数
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> # 在每个元素上调用一个方法
>>> freshfruit = ['  banana', '  loganberry ', 'passion fruit  ']
>>> [weapon.strip() for weapon in freshfruit]
['banana', 'loganberry', 'passion fruit']
>>> # 创建一个包含 (数字, 平方) 2 元组的列表
```

```
>>> [(x, x**2) for x in range(6)]
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
>>> # 元组必须加圆括号，否则会引发错误
>>> [x, x**2 for x in range(6)]
  File "<stdin>", line 1
    [x, x**2 for x in range(6)]
     ^^^^^^^
SyntaxError: did you forget parentheses around the comprehension target?
>>> # 使用两个 'for' 来展平嵌套的列表
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6, 7, 8, 9]

```

列表推导式可以使用复杂的表达式和嵌套函数：`>>>`

```
>>> from math import pi
>>> [str(round(pi, i)) for i in range(1, 6)]
['3.1', '3.14', '3.142', '3.1416', '3.14159']

```

5.1.4. 嵌套的列表推导式

列表推导式中的初始表达式可以是任何表达式，甚至可以是另一个列表推导式。

下面这个 3x4 矩阵，由 3 个长度为 4 的列表组成：`>>>`

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]

```

下面的列表推导式可以转置行列：`>>>`

```
>>> [[row[i] for row in matrix] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

```

如我们在之前小节中看到的，内部的列表推导式是在它之后的 for 的上下文中被求值的，所以这个例子等价于:`>>>`

```
>>> transposed = []
>>> for i in range(4):
...     transposed.append([row[i] for row in matrix])
...
>>> transposed
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

```

反过来说，也等价于：>>>

```
>>> transposed = []
>>> for i in range(4):
...     # 以下 3 行实现了嵌套的列表组
...     transposed_row = []
...     for row in matrix:
...         transposed_row.append(row[i])
...     transposed.append(transposed_row)
...
>>> transposed
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

```

实际应用中，最好用内置函数替代复杂的流程语句。此时，zip() 函数更好用：>>>

```
>>> list(zip(*matrix))
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]

```

关于本行中星号的详细说明，参见 解包实参列表。

5.2. del 语句

可以按索引而不是按值从一个列表移除条目：即使用 del 语句。 这不同于返回一个值的 pop() 方法。 del 语句还可被用来从列表移除切片或清空整个列表（之前我们通过将一个空列表赋值给切片实现此功能）。 例如:>>>

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
```

```
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]

```

del 也可以用来删除整个变量：`>>>`

```
>>> del a

```

此后，再引用 `a` 就会报错（直到为它赋与另一个值）。后文会介绍 del 的其他用法。

5.3. 元组和序列

列表和字符串有很多共性，例如，索引和切片操作。这两种数据类型是 **序列** （参见 序列类型 --- list, tuple, range）。随着 Python 语言的发展，其他的序列类型也被加入其中。本节介绍另一种标准序列类型：**元组**。

元组由多个用逗号隔开的值组成，例如：`>>>`

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> # 元组可以嵌套:
>>> u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> # 元组是不可变对象:
>>> t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # 但它们可以包含可变对象:
>>> v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])

```

输出时，元组都要由圆括号标注，这样才能正确地解释嵌套元组。输入时，圆括号可有可无，不过经常是必须的（如果元组是更大的表达式的一部分）。不允许为元组中的单个元素赋值，当然，可以创建含列表等可变对象的元组。
虽然，元组与列表很像，但使用场景不同，用途也不同。元组是 immutable （不可变的），一般可包含异质元素序列，通过解包（见本节下文）或索引访问（如果是 namedtuples，可以属性访问）。列表是 mutable （可变的），列表元素一般为同质类型，可迭代访问。
构造 0 个或 1 个元素的元组比较特殊：为了适应这种情况，对句法有一些额外的改变。用一对空圆括号就可以创建空元组；只有一个元素的元组可以通过在这个元素后添加逗号来构建（圆括号里只有一个值的话不够明确）。丑陋，但是有效。例如：>>>

```
>>> empty = ()
>>> singleton = 'hello',    # <-- 注意末尾的逗号
>>> len(empty)
0
>>> len(singleton)
1
>>> singleton
('hello',)

```

语句 `t = 12345, 54321, 'hello!'` 是 **元组打包** 的例子：值 `12345, 54321` 和 `'hello!'` 一起被打包进元组。
逆操作也可以：>>>

```
>>> x, y, z = t

```

称之为 **序列解包** 也是妥妥的，适用于右侧的任何序列。序列解包时，左侧变量与右侧序列元素的数量应相等。注意，多重赋值其实只是元组打包和序列解包的组合。

5.4. 集合

Python 还支持 **集合** 这种数据类型。集合是由不重复元素组成的无序容器。基本用法包括成员检测、消除重复元素。集合对象支持合集、交集、差集、对称差分等数学运算。
创建集合用花括号或 set() 函数。注意，创建空集合只能用 `set()`，不能用 `{}`，`{}` 创建的是空字典，下一小节介绍数据结构：字典。
以下是一些简单的示例>>>

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)                      # 显示重复项已被移除
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                 # 快速成员检测
```

```
True
>>> 'crabgrass' in basket
False

>>> # 演示针对两个单词中独有的字母进行集合运算
>>>
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                     # a 中独有的字母
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                                 # 存在于 a 中但不存在于 b 中的字母
{'r', 'd', 'b'}
>>> a | b                                 # 存在于 a 或 b 中或两者中皆有的字母
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                                 # 同时存在于 a 和 b 中的字母
{'a', 'c'}
>>> a ^ b                                 # 存在于 a 或 b 中但非两者中皆有的字母
{'r', 'd', 'b', 'm', 'z', 'l'}

```

与 列表推导式 类似，集合也支持推导式：`>>>`

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}

```

5.5. 字典

另一个常用的 Python 内置数据类型是 **字典** (参见 映射类型 --- dict)。 字典在其他编程语言中可能称为“联合内存”或“联合数组”。 与以连续整数为索引的序列不同，字典是以 **键** 来索引的，键可以是任何不可变类型；字符串和数字总是可以作为键。 元组在其仅包含字符串、数字或元组时也可以作为键；如果一个元组直接或间接地包含了任何可变对象，则不可以用作键。 你不能使用列表作为键，因为列表可使用索引赋值、切片赋值或 append() 和 extend() 等方法进行原地修改。

可以把字典理解为 **键值对** 的集合，但字典的键必须是唯一的。花括号 `{}` 用于创建空字典。另一种初始化字典的方式是，在花括号里输入逗号分隔的键值对，这也是字典的输出方式。

字典的主要用途是通过关键字存储、提取值。用 `del` 可以删除键值对。用已存在的关键字存储值，与该关键字关联的旧值会被取代。通过不存在的键提取值，则会报错。

对字典执行 list(d) 操作，返回该字典中所有键的列表，按插入次序排列（如需排序，请使用 sorted(d)）。检查字典里是否存在某个键，使用关键字 in。

以下是一些字典的简单示例：>>>

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False

```

dict() 构造函数可以直接用键值对序列创建字典：>>>

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}

```

字典推导式可以用任意键值表达式创建字典：>>>

```
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}

```

关键字是比较简单的字符串时，直接用关键字参数指定键值对更便捷：>>>

```
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'guido': 4127, 'jack': 4098}

```

5.6. 循环的技巧

当对字典执行循环时，可以使用 items() 方法同时提取键及其对应的值。 >>>

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():
...     print(k, v)
...
gallahad the pure
robin the brave

```

在序列中循环时，用 enumerate() 函数可以同时取出位置索引和对应的值： >>>

```
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
...
0 tic
1 tac
2 toe

```

同时循环两个或多个序列时，用 zip() 函数可以将其内的元素一一匹配： >>>

```
>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print('What is your {0}?  It is {1}.'.format(q, a))
...
What is your name?  It is lancelot.
What is your quest?  It is the holy grail.
What is your favorite color?  It is blue.

```

为了逆向对序列进行循环，可以求出欲循环的正向序列，然后调用 reversed() 函数： >>>

```
>>> for i in reversed(range(1, 10, 2)):
...     print(i)
...
9
```

```
7
5
3
1

```

按指定顺序循环序列，可以用 sorted() 函数，在不改动原序列的基础上，返回一个重新的序列：>>>

```
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> for i in sorted(basket):
...     print(i)
...
apple
apple
banana
orange
orange
pear

```

使用 set() 去除序列中的重复元素。使用 sorted() 加 set() 则按排序后的顺序，循环遍历序列中的唯一元素：>>>

```
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> for f in sorted(set(basket)):
...     print(f)
...
apple
banana
orange
pear

```

一般来说，在循环中修改列表的内容时，创建新列表比较简单，且安全：>>>

```
>>> import math
>>> raw_data = [56.2, float('NaN'), 51.7, 55.3, 52.5, float('NaN'), 47.8]
>>> filtered_data = []
>>> for value in raw_data:
...     if not math.isnan(value):
...         filtered_data.append(value)
...
```

```
>>> filtered_data
[56.2, 51.7, 55.3, 52.5, 47.8]

```

5.7. 深入条件控制

while 和 if 条件句不只可以进行比较，还可以使用任意运算符。
比较运算符 in 和 not in 用于执行确定一个值是否存在（或不存在）于某个容器中的成员检测。 运算符 is 和 is not 用于比较两个对象是否是同一个对象。 所有比较运算符的优先级都一样，且低于任何数值运算符。
比较操作支持链式操作。例如，a < b == c 校验 a 是否小于 b，且 b 是否等于 c。
比较操作可以用布尔运算符 and 和 or 组合，并且，比较操作（或其他布尔运算）的结果都可以用 not 取反。这些操作符的优先级低于比较操作符；not 的优先级最高， or 的优先级最低，因此，A and not B or C 等价于 (A and (not B)) or C。与其他运算符操作一样，此处也可以用圆括号表示想要的组合。
布尔运算符 and 和 or 是所谓的 **短路** 运算符：其参数从左至右求值，一旦可以确定结果，求值就会停止。例如，如果 A 和 C 为真，B 为假，那么 A and B and C 不会对 C 求值。用作普通值而不是布尔值时，短路运算符的返回值通常是最后一个求了值的参数。
还可以把比较运算或其它布尔表达式的结果赋值给变量，例如： >>>

```
>>> string1, string2, string3 = '', 'Trondheim', 'Hammer Dance'
>>> non_null = string1 or string2 or string3
>>> non_null
'Trondheim'

```

注意，Python 与 C 不同，在表达式内部赋值必须显式使用 海象运算符 :=。 这避免了 C 程序中常见的问题：要在表达式中写 == 时，却写成了 =。

5.8. 序列和其他类型的比较

序列对象可以与相同序列类型的其他对象比较。这种比较使用 **字典式** 顺序：首先，比较前两个对应元素，如果不相等，则可确定比较结果；如果相等，则比较之后的两个元素，以此类推，直到其中一个序列结束。如果要比较的两个元素本身是相同类型的序列，则递归地执行字典式顺序比较。如果两个序列中所有的对应元素都相等，则两个序列相等。如果一个序列是另一个的初始子序列，则较短的序列可被视为较小（较少）的序列。 对于字符串来说，字典式顺序使用 Unicode 码位序号排序单个字符。下面列出了一些比较相同类型序列的例子：

```
(1, 2, 3)              < (1, 2, 4)
```

```
[1, 2, 3]              < [1, 2, 4]
'ABC' < 'C' < 'Pascal' < 'Python'
(1, 2, 3, 4)           < (1, 2, 4)
(1, 2)                 < (1, 2, -1)
(1, 2, 3)             == (1.0, 2.0, 3.0)
(1, 2, ('aa', 'ab'))   < (1, 2, ('abc', 'a'), 4)

```

注意，当比较不同类型的对象时，只要待比较的对象提供了合适的比较方法，就可以使用 `<` 和 `>` 进行比较。例如，混合的数字类型通过数字值进行比较，所以，0 等于 0.0，等等。如果没有提供合适的比较方法，解释器不会随便给出一个比较结果，而是引发 TypeError 异常。

6. 模块

退出 Python 解释器后，再次进入时，之前在 Python 解释器中定义的函数和变量就丢失了。因此，编写较长程序时，最好用文本编辑器代替解释器，执行文件中的输入内容，这就是编写 **脚本** 。随着程序越来越长，为了方便维护，最好把脚本拆分成多个文件。编写脚本还一个好处，不同程序调用同一个函数时，不用把函数定义复制到各个程序。

为实现这些需求，Python 把各种定义存入一个文件，在脚本或解释器的交互式实例中使用。这个文件就是 **模块** ；模块中的定义可以 **导入** 到其他模块或 **主** 模块（在顶层和计算器模式下，执行脚本中可访问的变量集）。

模块是包含 Python 定义和语句的文件。其文件名是模块名加后缀名 `.py` 。在模块内部，通过全局变量 `__name__` 可以获取模块名（即字符串）。例如，用文本编辑器在当前目录下创建 `fibo.py` 文件，输入以下内容：

```python
# 斐波那契数列模块

def fib(n):    # 打印斐波那契数列直到 n
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):   # 返回斐波那契数列直到 n
    result = []
    a, b = 0, 1
    while a < n:
```

```
        result.append(a)
        a, b = b, a+b
    return result

```

现在，进入 Python 解释器，用以下命令导入该模块：>>>

```
>>> import fibo

```

此操作不会直接把 fibo 中定义的函数名称添加到当前 namespace 中（请参阅 Python 作用域和命名空间 了解详情）；它只是将模块名称 fibo 添加到那里。 使用该模块名称你可以访问其中的函数:>>>

```
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'

```

如果经常使用某个函数，可以把它赋值给局部变量：>>>

```
>>> fib = fibo.fib
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

6.1. 模块详解

模块包含可执行语句及函数定义。这些语句用于初始化模块，且仅在 import 语句 **第一次** 遇到模块名时执行。[1] (文件作为脚本运行时，也会执行这些语句。)

每个模块都有自己的私有命名空间，它会被用作模块中定义的所有函数的全局命名空间。 因此，模块作者可以在模块内使用全局变量而不必担心与用户的全局变量发生意外冲突。 另一方面，如果你知道要怎么做就可以通过与引用模块函数一样的标记法 modname.itemname 来访问一个模块的全局变量。

模块可以导入其他模块。 根据惯例可以将所有 import 语句都放在模块（或者也可以说是脚本）的开头但这并非强制要求。 如果被放置于一个模块的最高层级，则被导入的模块名称会被添加到该模块的全局命名空间。

还有一种 import 语句的变化形式可以将来自某个模块的名称直接导入到导入方模块的命名空间中。例如:>>>

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

这条语句不会将所导入的模块的名称引入到局部命名空间中（因此在本示例中，fibo 将是未定义的名称）。
还有一种变体可以导入模块内定义的所有名称：>>>

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

这种方式会导入所有不以下划线（_）开头的名称。大多数情况下，不要用这个功能，这种方式向解释器导入了一批未知的名称，可能会覆盖已经定义的名称。
注意，一般情况下，不建议从模块或包内导入 *，因为，这项操作经常让代码变得难以理解。不过，为了在交互式会话中少打几个字，这么用也没问题。
模块名后使用 as 时，直接把 as 后的名称与导入模块绑定。>>>

```
>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

与 import fibo 一样，这种方式也可以有效地导入模块，唯一的区别是，导入的名称是 fib。
from 中也可以使用这种方式，效果类似：>>>

```
>>> from fibo import fib as fibonacci
>>> fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

备注
为了保证运行效率，每次解释器会话只导入一次模块。如果更改了模块内容，必须重启解释器；仅交互测试一个模块时，也可以使用 importlib.reload()，例如 import importlib; importlib.reload(modulename)。

6.1.1. 以脚本方式执行模块

可以用以下方式运行 Python 模块：

```
python fibo.py <arguments>

```

这项操作将执行模块里的代码，和导入模块一样，但会把 `__name__` 赋值为 `"__main__"`。 也就是把下列代码添加到模块末尾：

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))

```

这个文件既能被用作脚本，又能被用作一个可供导入的模块，因为解析命令行参数的那两行代码只有在模块作为“main”文件执行时才会运行：

```
$ python fibo.py 50
0 1 1 2 3 5 8 13 21 34

```

当这个模块被导入到其它模块时，那两行代码不运行：`>>>`

```
>>> import fibo
>>>

```

这常用于为模块提供一个便捷的用户接口，或用于测试（把模块作为执行测试套件的脚本运行）。

6.1.2. 模块搜索路径

当导入一个名为 spam 的模块时，解释器首先会搜索具有该名称的内置模块。 这些模块的名称在 sys.builtin_module_names 中列出。 如果未找到，它将在变量 sys.path 所给出的目录列表中搜索名为 `spam.py` 的文件。 sys.path 是从这些位置初始化的:

- 被命令行直接运行的脚本所在的目录（或未指定文件时的当前目录）。

- PYTHONPATH

（目录列表，与 shell 变量 PATH 的语法一样）。

- 依赖于安装的默认值（按照惯例包括一个

`site-packages` 目录，由 site 模块处理）。

更多细节请参阅 sys.path 模块搜索路径的初始化。

备注

在支持符号链接的文件系统中，“被命令行直接运行的脚本所在的目录”是符号链接最终指向的目录。换句话说，符号链接所在的目录并 **没有** 被添加至模块搜索路径。

初始化后，Python 程序可以更改 sys.path。脚本所在的目录先于标准库所在的路径被搜索。这意味着，脚本所在的目录如果有和标准库同名的文件，那么加载的是该目录里的，而不是标准库的。这一般是一个错误，除非这样的替换是你有意为之。详见 标准模块。

6.1.3. “已编译的” Python 文件

为了快速加载模块，Python 把模块的编译版本缓存在 __pycache__ 目录中，文件名为 module.**version**.pyc，version 对编译文件格式进行编码，一般是 Python 的版本号。例如，CPython 的 3.3 发行版中，spam.py 的编译版本缓存为 __pycache__/spam.cpython-33.pyc。这种命名惯例让不同 Python 版本编译的模块可以共存。

Python 对比编译版与源码的修改日期，查看编译版是否已过期，是否要重新编译。此进程完全是自动的。此外，编译模块与平台无关，因此，可在不同架构的系统之间共享相同的库。

Python 在两种情况下不检查缓存。一，从命令行直接载入的模块，每次都会重新编译，且不储存编译结果；二，没有源模块，就不会检查缓存。为了让一个库能以隐藏源代码的形式分发（通过将所有源代码变为编译后的版本），编译后的模块必须放在源目录而非缓存目录中，并且源目录绝不能包含同名的未编译的源模块。

给专业人士的一些小建议：

- 在 Python 命令中使用

-O 或 -OO 开关，可以减小编译模块的大小。-O 去除断言语句，-OO 去除断言语句和 __doc__ 字符串。有些程序可能依赖于这些内容，因此，没有十足的把握，不要使用这两个选项。“优化过的”模块带有 opt- 标签，并且文件通常会一小些。将来的发行版或许会改进优化的效果。

- 从

.pyc 文件读取的程序不比从 .py 读取的执行速度快，.pyc 文件只是加载速度更快。

- compileall

模块可以为一个目录下的所有模块创建 .pyc 文件。

- 本过程的细节及决策流程图，详见

PEP 3147。

6.2. 标准模块

Python 自带一个标准模块的库，它在 Python 库参考（此处以下称为"库参考"）里另外描述。 一些模块是内嵌到解释器里面的， 它们给一些虽并非语言核心但却内嵌的操作提供接口，要么是为了效率，要么是给操作系统基础操作例如系统调入提供接口。 这些模块集是一个配置选项， 并且还依赖于底层的操作系统。 例如，winreg 模块只在 Windows 系统上提供。一个特别值得注意的模块 sys，它

被内嵌到每一个 Python 解释器中。sys.ps1 和 sys.ps2 变量定义了一些字符，它们可以用作主提示符和辅助提示符:>>>

```
>>> import sys
>>> sys.ps1
'>>> '
>>> sys.ps2
'... '
>>> sys.ps1 = 'C> '
C> print('Yuck!')
Yuck!
C>

```

只有解释器用于交互模式时，才定义这两个变量。
变量 sys.path 是字符串列表，用于确定解释器的模块搜索路径。该变量以环境变量 PYTHONPATH 提取的默认路径进行初始化，如未设置 PYTHONPATH，则使用内置的默认路径。可以用标准列表操作修改该变量：>>>

```
>>> import sys
>>> sys.path.append('/ufs/guido/lib/python')

```

6.3. dir() 函数

内置函数 dir() 用于查找模块定义的名称。返回结果是经过排序的字符串列表：>>>

```
>>> import fibo, sys
>>> dir(fibo)
['__name__', 'fib', 'fib2']
>>> dir(sys)
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthook__',
 '__interactivehook__', '__loader__', '__name__', '__package__', '__spec__',
 '__stderr__', '__stdin__', '__stdout__', '__unraisablehook__',
 '_clear_type_cache', '_current_frames', '_debugmallocstats', '_framework',
 '_getframe', '_git', '_home', '_xoptions', 'abiflags', 'addaudithook',
 'api_version', 'argv', 'audit', 'base_exec_prefix', 'base_prefix',
 'breakpointhook', 'builtin_module_names', 'byteorder', 'call_tracing',
 'callstats', 'copyright', 'displayhook', 'dont_write_bytecode', 'exc_info',
```

```
 'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'float_info',
 'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_depth',
 'getallocatedblocks', 'getdefaultencoding', 'getdlopenflags',
 'getfilesystemencodeerrors', 'getfilesystemencoding', 'getprofile',
 'getrecursionlimit', 'getrefcount', 'getsizeof', 'getswitchinterval',
 'gettrace', 'hash_info', 'hexversion', 'implementation', 'int_info',
 'intern', 'is_finalizing', 'last_traceback', 'last_type', 'last_value',
 'maxsize', 'maxunicode', 'meta_path', 'modules', 'path', 'path_hooks',
 'path_importer_cache', 'platform', 'prefix', 'ps1', 'ps2', 'pycache_prefix',
 'set_asyncgen_hooks', 'set_coroutine_origin_tracking_depth', 'setdlopenflags',
 'setprofile', 'setrecursionlimit', 'setswitchinterval', 'settrace', 'stderr',
 'stdin', 'stdout', 'thread_info', 'unraisablehook', 'version', 'version_info',
 'warnoptions']

```

没有参数时，dir() 列出当前已定义的名称：>>>

```
>>> a = [1, 2, 3, 4, 5]
>>> import fibo
>>> fib = fibo.fib
>>> dir()
['__builtins__', '__name__', 'a', 'fib', 'fibo', 'sys']

```

注意它列出所有类型的名称：变量，模块，函数，……。
dir() 不会列出内置函数和变量的名称。这些内容的定义在标准模块 builtins 中：>>>

```
>>> import builtins
>>> dir(builtins)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning',
 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False',
 'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
 'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented',
 'NotImplementedError', 'OSError', 'OverflowError',
 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
```

```
'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError',
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'ZeroDivisionError', '_', '__build_class__',
'__debug__', '__doc__', '__import__', '__name__', '__package__', 'abs',
'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable',
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit',
'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr',
'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass',
'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview',
'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice',
'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars',
'zip']

```

6.4. 包

包是通过使用“带点号模块名”来构造 Python 模块命名空间的一种方式。 例如，模块名 A.B 表示名为 A 的包中名为 B 的子模块。 就像使用模块可以让不同模块的作者不必担心彼此的全局变量名一样，使用带点号模块名也可以让 NumPy 或 Pillow 等多模块包的作者也不必担心彼此的模块名冲突。

假设要为统一处理声音文件与声音数据设计一个模块集（“包”）。声音文件的格式很多（通常以扩展名来识别，例如：.wav，.aiff，.au），因此，为了不同文件格式之间的转换，需要创建和维护一个不断增长的模块集合。为了实现对声音数据的不同处理（例如，混声、添加回声、均衡器功能、创造人工立体声效果），还要编写无穷无尽的模块流。下面这个分级文件树展示了这个包的架构：

```
sound/                          最高层级的包
      __init__.py               初始化 sound 包
      formats/                  用于文件格式转换的子包
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
```

```
                auwrite.py
                ...
        effects/                    用于音效的子包
                __init__.py
                echo.py
                surround.py
                reverse.py
                ...
        filters/                    用于过滤器的子包
                __init__.py
                equalizer.py
                vocoder.py
                karaoke.py
                ...

```

导入包时，Python 搜索 sys.path 里的目录，查找包的子目录。
需要有 __init__.py 文件才能让 Python 将包含该文件的目录当作包来处理（除非使用 namespace package，这是一个相对高级的特性）。 这可以防止重名的目录如 string 在无意中屏蔽后继出现在模块搜索路径中的有效模块。 在最简单的情况下，__init__.py 可以只是一个空文件，但它也可以执行包的初始化代码或设置 __all__ 变量，这将在稍后详细描述。
还可以从包中导入单个模块，例如：

```
import sound.effects.echo

```

这将加载子模块 sound.effects.echo。 它必须通过其全名来引用。

```
sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)

```

另一种导入子模块的方法是：

```
from sound.effects import echo

```

这也会加载子模块 echo，并使其不必加包前缀，因此可按如下方式使用:

```
echo.echofilter(input, output, delay=0.7, atten=4)

```

Import 语句的另一种变体是直接导入所需的函数或变量：

```
from sound.effects.echo import echofilter

```

同样，这将加载子模块 echo，但这使其函数 echofilter() 直接可用:

```
echofilter(input, output, delay=0.7, atten=4)

```

注意，使用 from package import item 时，item 可以是包的子模块（或子包），也可以是包中定义的函数、类或变量等其他名称。import 语句首先测试包中是否定义了 item；如果未在包中定义，则假定 item 是模块，并尝试加载。如果找不到 item，则触发 ImportError 异常。
相反，使用 import item.subitem.subsubitem 句法时，除最后一项外，每个 item 都必须是包；最后一项可以是模块或包，但不能是上一项中定义的类、函数或变量。

6.4.1. 从包中导入 *

使用 from sound.effects import * 时会发生什么？你可能希望它会查找并导入包的所有子模块，但事实并非如此。因为这将花费很长的时间，并且可能会产生你不想要的副作用，如果这种副作用被你设计为只有在导入某个特定的子模块时才应该发生。
唯一的解决办法是提供包的显式索引。import 语句使用如下惯例：如果包的 __init__.py 代码定义了列表 __all__，运行 from package import * 时，它就是被导入的模块名列表。发布包的新版本时，包的作者应更新此列表。如果包的作者认为没有必要在包中执行导入 * 操作，也可以不提供此列表。例如，sound/effects/__init__.py 文件可以包含以下代码：

```
__all__ = ["echo", "surround", "reverse"]

```

这意味着 from sound.effects import * 将导入 sound.effects 包的三个命名子模块。
请注意子模块可能会受到本地定义名称的影响。 例如，如果你在 sound/effects/__init__.py 文件中添加了一个 reverse 函数，from sound.effects import * 将只导入 echo 和 surround 这两个子模块，但 **不会** 导入 reverse 子模块，因为它被本地定义的 reverse 函数所遮挡:

```
__all__ = [
    "echo",       # 指向 'echo.py' 文件
    "surround",   # 指向 'surround.py' 文件
    "reverse",    # !!! 现在指向 'reverse' 函数 !!!
]

def reverse(msg: str):  # <-- 此名称将覆盖 'reverse.py' 子模块
    return msg[::-1]    #     针对 'from sound.effects import *' 的情况

```

如果没有定义 `__all__`，`from sound.effects import *` 语句 **不会** 把包 sound.effects 中的所有子模块都导入到当前命名空间；它只是确保包 sound.effects 已被导入（可能还会运行 `__init__.py` 中的任何初始化代码），然后再导入包中定义的任何名称。 这包括由 `__init__.py` 定义的任何名称（以及显式加载的子模块）。 它还包括先前 import 语句显式加载的包里的任何子模块。 请看以下代码:

```
import sound.effects.echo
import sound.effects.surround
from sound.effects import *

```

在本例中，echo 和 surround 模块被导入到当前命名空间，因为在执行 `from...import` 语句时它们已在 sound.effects 包中定义了。 （当定义了 `__all__` 时也是如此）。
虽然，可以把模块设计为用 `import *` 时只导出遵循指定模式的名称，但仍不提倡在生产代码中使用这种做法。
记住，使用 `from package import specific_submodule` 没有任何问题！ 实际上，除了导入模块使用不同包的同名子模块之外，这种方式是推荐用法。

6.4.2. 相对导入

当包由多个子包构成（如示例中的 sound 包）时，可以使用绝对导入来引用同级包的子模块。 例如，如果 sound.filters.vocoder 模块需要使用 sound.effects 包中的 echo 模块，它可以使用 `from sound.effects import echo`。
你还可以编写相对导入代码，即使用 `from module import name` 形式的 import 语句。 这些导入使用前导点号来表示相对导入所涉及的当前包和上级包。 例如对于 surround 模块，可以使用:

```
from . import echo
from .. import formats
from ..filters import equalizer

```

注意，相对导入基于当前模块名。因为主模块名永远是 `"__main__"`，所以如果计划将一个模块用作 Python 应用程序的主模块，那么该模块内的导入语句必须始终使用绝对导入。

6.4.3. 多目录中的包

包还支持一个特殊的属性， `__path__` 。 在执行该文件中的代码之前，它被初始化为字符串的 sequence，其中包含包的 `__init__.py` 的目录名称。这个变量可以修改；修改后会影响今后对模块和包中包含的子包的搜索。
这个功能虽然不常用，但可用于扩展包中的模块集。

7. 输入与输出

程序输出有几种显示方式；数据既可以输出供人阅读的形式，也可以写入文件备用。本章探讨一些可用的方式。

7.1. 更复杂的输出格式

到目前为止我们已遇到过两种写入值的方式: **表达式语句** 和 print() 函数。 （第三种方式是使用文件对象的 write() 方法；标准输出文件可以被引用为 sys.stdout。 更多相关信息请参阅标准库参考）。

对输出格式的控制不只是打印空格分隔的值，还需要更多方式。格式化输出包括以下几种方法。

- 使用

格式化字符串字面值 ，要在字符串开头的引号/三引号前添加 f 或 F 。在这种字符串中，可以在 { 和 } 字符之间输入引用的变量，或字面值的 Python 表达式。 >>>

```
>>> year = 2016
>>> event = 'Referendum'
>>> f'Results of the {year} {event}'
'Results of the 2016 Referendum'

```

- 字符串的

str.format() 方法需要更多手动操作。 你仍将使用 { 和 } 来标记变量将被替换的位置并且可以提供详细的格式化指令，但你还需要提供待格式化的信息。 下面的代码块中有两个格式化变量的例子： >>>

```
>>> yes_votes = 42_572_654
>>> total_votes = 85_705_149
>>> percentage = yes_votes / total_votes
>>> '{:-9} YES votes  {:2.2%}'.format(yes_votes, percentage)
' 42572654 YES votes  49.67%'

```

请注意Notice how the yes_votes 填充了空格并且只为负数添加了负号。 这个例子还打印了 percentage 乘以 100 的结果，保留 2 个数位并带有一个百分号 (请参阅 格式规格迷你语言 了解详情)。

- 最后，还可以用字符串切片和合并操作完成字符串处理操作，创建任何排版布局。字符串类型还支持将字符串按给定列宽进行填充，这些方法也很有用。

如果不需要花哨的输出，只想快速显示变量进行调试，可以用 repr() 或 str() 函数把值转化为字符串。

str() 函数返回供人阅读的值，repr() 则生成适于解释器读取的值（如果没有等效的语法，则强制执行 SyntaxError）。对于没有支持供人阅读展示结果的对象， str() 返回与 repr() 相同的值。一般情况下，数字、列表或字典等结构的值，使用这两个函数输出的表现形式是一样的。字符串有两种不同的表现形式。

示例如下：`>>>`

```
>>> s = 'Hello, world.'
>>> str(s)
'Hello, world.'
>>> repr(s)
"'Hello, world.'"
>>> str(1/7)
'0.14285714285714285'
>>> x = 10 * 3.25
>>> y = 200 * 200
>>> s = 'The value of x is ' + repr(x) + ', and y is ' + repr(y) + '...'
>>> print(s)
The value of x is 32.5, and y is 40000...
>>> # The repr() of a string adds string quotes and backslashes:
>>> hello = 'hello, world\n'
>>> hellos = repr(hello)
>>> print(hellos)
'hello, world\n'
>>> # The argument to repr() may be any Python object:
>>> repr((x, y, ('spam', 'eggs')))
"(32.5, 40000, ('spam', 'eggs'))"

```

string 模块包含 Template 类，提供了将值替换为字符串的另一种方法。该类使用 `$x` 占位符，并用字典的值进行替换，但对格式控制的支持比较有限。

7.1.1. 格式化字符串字面值

格式化字符串字面值 （简称为 f-字符串）在字符串前加前缀 `f` 或 `F`，通过 `{expression}` 表达式，把 Python 表达式的值添加到字符串内。

格式说明符是可选的，写在表达式后面，可以更好地控制格式化值的方式。下例将 pi 舍入到小数点后三位：`>>>`

```
>>> import math
>>> print(f'The value of pi is approximately {math.pi:.3f}.')
```

```
The value of pi is approximately 3.142.

```

在 ':' 后传递整数，为该字段设置最小字符宽度，常用于列对齐： >>>

```
>>> table = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 7678}
>>> for name, phone in table.items():
...     print(f'{name:10} ==> {phone:10d}')
...
Sjoerd     ==>       4127
Jack       ==>       4098
Dcab       ==>       7678

```

还有一些修饰符可以在格式化前转换值。 '!a' 应用 ascii() ， '!s' 应用 str()， '!r' 应用 repr()： >>>

```
>>> animals = 'eels'
>>> print(f'My hovercraft is full of {animals}.')
My hovercraft is full of eels.
>>> print(f'My hovercraft is full of {animals!r}.')
My hovercraft is full of 'eels'.

```

= 说明符可被用于将一个表达式扩展为表达式文本、等号再加表达式求值结果的形式。 >>>

```
>>> bugs = 'roaches'
>>> count = 13
>>> area = 'living room'
>>> print(f'Debugging {bugs=} {count=} {area=}')
Debugging bugs='roaches' count=13 area='living room'

```

请参阅 自说明型表达式 以了解 = 说明符的更多信息。 有关这些格式说明的详情，请查看针对 格式规格迷你语言 的参考指南。

7.1.2. 字符串 format() 方法

str.format() 方法的基本用法如下所示： >>>

```
>>> print('We are the {} who say "{}!"'.format('knights', 'Ni'))
We are the knights who say "Ni!"

```

花括号及之内的字符（称为格式字段）被替换为传递给 str.format() 方法的对象。花括号中的数字表示传递给 str.format() 方法的对象所在的位置。 >>>

```
>>> print('{0} and {1}'.format('spam', 'eggs'))
spam and eggs
>>> print('{1} and {0}'.format('spam', 'eggs'))
eggs and spam

```

str.format() 方法中使用关键字参数名引用值。 >>>

```
>>> print('This {food} is {adjective}.'.format(
...       food='spam', adjective='absolutely horrible'))
This spam is absolutely horrible.

```

位置参数和关键字参数可以任意组合： >>>

```
>>> print('The story of {0}, {1}, and {other}.'.format('Bill', 'Manfred',
...                                                    other='Georg'))
The story of Bill, Manfred, and Georg.

```

如果不想分拆较长的格式字符串，最好按名称引用变量进行格式化，不要按位置。这项操作可以通过传递字典，并用方括号 '[]' 访问键来完成。 >>>

```
>>> table = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 8637678}
>>> print('Jack: {0[Jack]:d}; Sjoerd: {0[Sjoerd]:d}; '
...       'Dcab: {0[Dcab]:d}'.format(table))
Jack: 4098; Sjoerd: 4127; Dcab: 8637678

```

这也可以通过将 table 字典作为采用 ** 标记的关键字参数传入来实现。 >>>

```
>>> table = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 8637678}
>>> print('Jack: {Jack:d}; Sjoerd: {Sjoerd:d}; Dcab: {Dcab:d}'.format(**table))
Jack: 4098; Sjoerd: 4127; Dcab: 8637678

```

与内置函数 vars() 一同使用时这种方式非常实用，它将返回一个包含所有局部变量的字典:>>>

```
>>> table = {k: str(v) for k, v in vars().items()}
>>> message = " ".join([f'{k}: ' + '{' + k +'};' for k in table.keys()])
```

```
>>> print(message.format(**table))
__name__: __main__; __doc__: None; __package__: None; __loader__: ...

```

举个例子，以下几行代码将产生一组整齐的数据列，包含给定的整数及其平方与立方:>>>

```
>>> for x in range(1, 11):
...     print('{0:2d} {1:3d} {2:4d}'.format(x, x*x, x*x*x))
...
 1   1    1
 2   4    8
 3   9   27
 4  16   64
 5  25  125
 6  36  216
 7  49  343
 8  64  512
 9  81  729
10 100 1000

```

str.format() 进行字符串格式化的完整概述详见 格式字符串语法 。

7.1.3. 手动格式化字符串

下面是使用手动格式化方式实现的同一个平方和立方的表： >>>

```
>>> for x in range(1, 11):
...     print(repr(x).rjust(2), repr(x*x).rjust(3), end=' ')
...     # 请注意上一行中 'end' 的使用
...     print(repr(x*x*x).rjust(4))
...
 1   1    1
 2   4    8
 3   9   27
 4  16   64
 5  25  125
 6  36  216
 7  49  343
 8  64  512
 9  81  729
10 100 1000
```

```

```

（注意，每列之间的空格是通过使用 print() 添加的：它总在其参数间添加空格。）
字符串对象的 str.rjust() 方法通过在左侧填充空格，对给定宽度字段中的字符串进行右对齐。同类方法还有 str.ljust() 和 str.center() 。这些方法不写入任何内容，只返回一个新字符串，如果输入的字符串太长，它们不会截断字符串，而是原样返回；虽然这种方式会弄乱列布局，但也比另一种方法好，后者在显示值时可能不准确（如果真的想截断字符串，可以使用 x.ljust(n)[:n] 这样的切片操作 。）
另一种方法是 str.zfill() ，该方法在数字字符串左边填充零，且能识别正负号：>>>

```
>>> '12'.zfill(5)
'00012'
>>> '-3.14'.zfill(7)
'-003.14'
>>> '3.14159265359'.zfill(5)
'3.14159265359'

```

7.1.4. 旧式字符串格式化方法

% 运算符 (求余) 也可被用于字符串格式化。 给定 format % values (其中 **format** 是一个字符串)，则 **format** 中的 % 转换占位符将以 **values** 中的零个或多个元素来替换。 此操作通常称为字符串插值。 例如:>>>

```
>>> import math
>>> print('The value of pi is approximately %5.3f.' % math.pi)
The value of pi is approximately 3.142.

```

printf 风格的字符串格式化 小节介绍更多相关内容。

7.2. 读写文件

open() 返回一个 file object ， 最常使用的是两个位置参数和一个关键字参数：open(filename, mode, encoding=None)>>>

```
>>> f = open('workfile', 'w', encoding="utf-8")

```

第一个实参是文件名字符串。第二个实参是包含描述文件使用方式字符的字符串。**mode** 的值包括 'r' ，表示文件只能读取；'w' 表示只能写入（现有同名文件会被覆盖）；'a' 表示打开文件并追加内

容，任何写入的数据会自动添加到文件末尾。'r+' 表示打开文件进行读写。**mode** 实参是可选的，省略时的默认值为 'r'。
通常情况下，文件是以 **text mode** 打开的，也就是说，你从文件中读写字符串，这些字符串是以特定的 **encoding** 编码的。如果没有指定 **encoding** ，默认的是与平台有关的（见 open() ）。因为 UTF-8 是现代事实上的标准，除非你知道你需要使用一个不同的编码，否则建议使用 encoding="utf-8" 。
在模式后面加上一个 'b' ，可以用 **binary mode** 打开文件。二进制模式的数据是以 bytes 对象的形式读写的。在二进制模式下打开文件时，你不能指定 **encoding** 。
在文本模式下读取文件时，默认把平台特定的行结束符（Unix 上为 \n, Windows 上为 \r\n）转换为 \n。在文本模式下写入数据时，默认把 \n 转换回平台特定结束符。这种操作方式在后台修改文件数据对文本文件来说没有问题，但会破坏 JPEG 或 EXE 等二进制文件中的数据。注意，在读写此类文件时，一定要使用二进制模式。
在处理文件对象时，最好使用 with 关键字。优点是，子句体结束后，文件会正确关闭，即便触发异常也可以。而且，使用 with 相比等效的 try-finally 代码块要简短得多：>>>

```
>>> with open('workfile', encoding="utf-8") as f:
...     read_data = f.read()

>>> # 我们可以检测文件是否已被自动关闭。
>>> f.closed
True

```

如果没有使用 with 关键字，则应调用 f.close() 关闭文件，即可释放文件占用的系统资源。
警告
调用 f.write() 时，未使用 with 关键字，或未调用 f.close()，即使程序正常退出，也**可能** 导致 f.write() 的参数没有完全写入磁盘。
通过 with 语句，或调用 f.close() 关闭文件对象后，再次使用该文件对象将会失败。>>>

```
>>> f.close()
>>> f.read()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: I/O operation on closed file.

```

7.2.1. 文件对象的方法

本节下文中的例子假定已创建 f 文件对象。

f.read(size) 可用于读取文件内容，它会读取一些数据，并返回字符串（文本模式），或字节串对象（在二进制模式下）。 **size** 是可选的数值参数。省略 **size** 或 **size** 为负数时，读取并返回整个文件的内容；文件大小是内存的两倍时，会出现问题。**size** 取其他值时，读取并返回最多 **size** 个字符（文本模式）或 **size** 个字节（二进制模式）。如已到达文件末尾，f.read() 返回空字符串（''）。 >>>

```
>>> f.read()
'This is the entire file.\n'
>>> f.read()
''

```

f.readline() 从文件中读取单行数据；字符串末尾保留换行符（\n），只有在文件不以换行符结尾时，文件的最后一行才会省略换行符。这种方式让返回值清晰明确；只要 f.readline() 返回空字符串，就表示已经到达了文件末尾，空行使用 '\n' 表示，该字符串只包含一个换行符。 >>>

```
>>> f.readline()
'This is the first line of the file.\n'
>>> f.readline()
'Second line of the file\n'
>>> f.readline()
''

```

从文件中读取多行时，可以用循环遍历整个文件对象。这种操作能高效利用内存，快速，且代码简单： >>>

```
>>> for line in f:
...     print(line, end='')
...
This is the first line of the file.
Second line of the file

```

如需以列表形式读取文件中的所有行，可以用 list(f) 或 f.readlines()。
f.write(string) 把 **string** 的内容写入文件，并返回写入的字符数。 >>>

```
>>> f.write('This is a test\n')
15

```

写入其他类型的对象前，要先把它们转化为字符串（文本模式）或字节对象（二进制模式）： >>>

```
>>> value = ('the answer', 42)
>>> s = str(value)  # 将元组转换为字符串
>>> f.write(s)
18

```

f.tell() 返回整数，给出文件对象在文件中的当前位置，表示为二进制模式下时从文件开始的字节数，以及文本模式下的意义不明的数字。
f.seek(offset, whence) 可以改变文件对象的位置。通过向参考点添加 **offset** 计算位置；参考点由 **whence** 参数指定。 **whence** 值为 0 时，表示从文件开头计算，1 表示使用当前文件位置，2 表示使用文件末尾作为参考点。省略 **whence** 时，其默认值为 0，即使用文件开头作为参考点。 >>>

```
>>> f = open('workfile', 'rb+')
>>> f.write(b'0123456789abcdef')
16
>>> f.seek(5)      # 定位到文件中的第 6 个字节
5
>>> f.read(1)
b'5'
>>> f.seek(-3, 2)  # 定位到倒数第 3 个字节
13
>>> f.read(1)
b'd'

```

在文本文件（模式字符串未使用 b 时打开的文件）中，只允许相对于文件开头搜索（使用 seek(0, 2) 搜索到文件末尾是个例外），唯一有效的 **offset** 值是能从 f.tell() 中返回的，或 0。其他 **offset** 值都会产生未定义的行为。
文件对象还有一些额外的方法，如使用频率较低的 isatty() 和 truncate() 等；有关文件对象的完整指南请查阅标准库参考。

7.2.2. 使用 json 保存结构化数据

字符串可以很容易地写入文件或从文件中读取。 数字则更麻烦一些，因为 read() 方法只返回字符串，而字符串必须传给 int() 这样的函数，它接受 '123' 这样的字符串并返回其数值 123。 当你想要保存嵌套列表和字典等更复杂的数据类型时，手动执行解析和序列化操作将会变得非常复杂。
Python 允许你使用流行的数据交换格式 JSON (JavaScript Object Notation)，而不是让用户持续编写和调试代码来将复杂的数据类型存入文件中。 标准库模块 json 可以接受带有层级结构的 Python 数据，并将其转换为字符串表示形式；这个过程称为 **serializing**。 根据字符串表示形式重建数据则称

为 **deserializing**。 在序列化和反序列化之间，用于代表对象的字符串可以存储在文件或数据库中，或者通过网络连接发送到远端主机。

备注

JSON 格式通常用于现代应用程序的数据交换。程序员早已对它耳熟能详，可谓是交互操作的不二之选。

只需一行简单的代码即可查看某个对象的 JSON 字符串表现形式：`>>>`

```
>>> import json
>>> x = [1, 'simple', 'list']
>>> json.dumps(x)
'[1, "simple", "list"]'

```

dumps() 函数还有一个变体， dump() ，它只将对象序列化为 text file 。因此，如果 `f` 是 text file 对象，可以这样做：

```
json.dump(x, f)

```

要再次解码对象，如果 `f` 是已打开、供读取的 binary file 或 text file 对象：

```
x = json.load(f)

```

8. 错误和异常

至此，本教程还未深入介绍错误信息，但如果您尝试过本教程前文中的例子，应该已经看到过一些错误信息。错误可（至少）被分为两种：**语法错误** 和 **异常**。

8.1. 语法错误

语法错误又称解析错误，是学习 Python 时最常见的错误：`>>>`

```
>>> while True print('Hello world')
  File "<stdin>", line 1
    while True print('Hello world')
               ^^^^^
SyntaxError: invalid syntax

```

解析器会重复存在错误的行并显示一个指向廖行中检测到错误的词元的小箭头。 错误可能是由于所指向的词元 **之前** 缺少某个词元而导致的。 在这个例子中，错误是在函数 print() 上检测到的，原因是在它之前缺少一个冒号 (':')。 文件名和行号也会被打印出来以便你在输入是来自脚本时可以知道要去哪里查找问题。

8.2. 异常

即使语句或表达式使用了正确的语法，执行时仍可能触发错误。执行时检测到的错误称为 **异常**，异常不一定导致严重的后果：很快我们就能学会如何处理 Python 的异常。大多数异常不会被程序处理，而是显示下列错误信息：`>>>`

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
          ~^~
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    4 + spam*3
        ^^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    '2' + 2
    ~~~~^~~
TypeError: can only concatenate str (not "int") to str

```

错误信息的最后一行说明程序遇到了什么类型的错误。异常有不同的类型，而类型名称会作为错误信息的一部分中打印出来：上述示例中的异常类型依次是：
ZeroDivisionError， NameError 和 TypeError。作为异常类型打印的字符串是发生的内置异常的名称。对于所有内置异常都是如此，但对于用户定义的异常则不一定如此（虽然这种规范很有用）。标准的异常类型是内置的标识符（不是保留关键字）。
此行其余部分根据异常类型，结合出错原因，说明错误细节。
错误信息开头用堆栈回溯形式展示发生异常的语境。一般会列出源代码行的堆栈回溯；但不会显示从标准输入读取的行。

内置异常 列出了内置异常及其含义。

8.3. 异常的处理

可以编写程序处理选定的异常。下例会要求用户一直输入内容，直到输入有效的整数，但允许用户中断程序（使用 Control-C 或操作系统支持的其他操作）；注意，用户中断程序会触发 KeyboardInterrupt 异常。 >>>

```
>>> while True:
...     try:
...         x = int(input("Please enter a number: "))
...         break
...     except ValueError:
...         print("Oops!  That was no valid number.  Try again...")
...

```

try 语句的工作原理如下：

- 首先，执行

try 子句 （try 和 except 关键字之间的（多行）语句）。

- 如果没有触发异常，则跳过

except 子句，try 语句执行完毕。

- 如果在执行

try 子句时发生了异常，则跳过该子句中剩下的部分。 如果异常的类型与 except 关键字后指定的异常相匹配，则会执行 **except 子句**，然后跳到 try/except 代码块之后继续执行。

- 如果发生的异常与

except 子句 中指定的异常不匹配，则它会被传递到外层的 try 语句中；如果没有找到处理器，则它是一个 **未处理异常** 且执行将停止并输出一条错误消息。

try 语句可以有多个 **except 子句** 来为不同的异常指定处理程序。 但最多只有一个处理程序会被执行。 处理程序只处理对应的 **try 子句** 中发生的异常，而不处理同一 try 语句内其他处理程序中的异常。 **except 子句** 可以用带圆括号的元组来指定多个异常，例如:

```
... except (RuntimeError, TypeError, NameError):
...     pass

```

一个 except 子句中的类匹配的异常将是该类本身的实例或其所派生的类的实例（但反过来则不可以 --- 列出派生类的 **except 子句** 不会匹配其基类的实例）。 例如，下面的代码将依次打印 B, C, D:

```
class B(Exception):
```

```
    pass

class C(B):
    pass

class D(C):
    pass

for cls in [B, C, D]:
    try:
        raise cls()
    except D:
        print("D")
    except C:
        print("C")
    except B:
        print("B")

```

请注意如果颠倒 **except 子句** 的顺序（把 except B 放在最前），则会输出 B, B, B --- 即触发了第一个匹配的 **except 子句**。

发生异常时，它可能具有关联值，即异常 **参数** 。是否需要参数，以及参数的类型取决于异常的类型。 **except 子句** 可能会在异常名称后面指定一个变量。 这个变量将被绑定到异常实例，该实例通常会有一个存储参数的 args 属性。 为了方便起见，内置异常类型定义了 __str__() 来打印所有参数而不必显式地访问 .args。 >>>

```
>>> try:
...     raise Exception('spam', 'eggs')
... except Exception as inst:
...     print(type(inst))    # 异常的类型
...     print(inst.args)     # 参数保存在 .args 中
...     print(inst)          # __str__ 允许 args 被直接打印，
...                          # 但可能在异常子类中被覆盖
...     x, y = inst.args     # 解包 args
...     print('x =', x)
...     print('y =', y)
...
<class 'Exception'>
('spam', 'eggs')
('spam', 'eggs')
```

```
x = spam
y = eggs

```

未处理异常的 __str__() 输出会被打印为该异常消息的最后部分 ('detail')。
BaseException 是所有异常的共同基类。它的一个子类， Exception ，是所有非致命异常的基类。不是 Exception 的子类的异常通常不被处理，因为它们被用来指示程序应该终止。它们包括由 sys.exit() 引发的 SystemExit ，以及当用户希望中断程序时引发的 KeyboardInterrupt 。
Exception 可以被用作通配符，捕获（几乎）一切。然而，好的做法是，尽可能具体地说明我们打算处理的异常类型，并允许任何意外的异常传播下去。
处理 Exception 最常见的模式是打印或记录异常，然后重新提出（允许调用者也处理异常）：

```
import sys

try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("OS error:", err)
except ValueError:
    print("Could not convert data to an integer.")
except Exception as err:
    print(f"Unexpected {err=}, {type(err)=}")
    raise

```

try … except 语句具有可选的 **else 子句**，该子句如果存在，它必须放在所有 **except 子句** 之后。 它适用于 **try 子句** 没有引发异常但又必须要执行的代码。 例如:

```
for arg in sys.argv[1:]:
    try:
        f = open(arg, 'r')
    except OSError:
        print('cannot open', arg)
    else:
        print(arg, 'has', len(f.readlines()), 'lines')
        f.close()

```

使用 else 子句比向 try 子句添加额外的代码要好，可以避免意外捕获非 try ... except 语句保护的代码触发的异常。
异常处理程序不仅会处理在 **try 子句** 中立刻发生的异常，还会处理在 **try 子句** 中调用（包括间接调用）的函数。 例如:>>>

```
>>> def this_fails():
...     x = 1/0
...
>>> try:
...     this_fails()
... except ZeroDivisionError as err:
...     print('Handling run-time error:', err)
...
Handling run-time error: division by zero

```

8.4. 触发异常

raise 语句支持强制触发指定的异常。例如： >>>

```
>>> raise NameError('HiThere')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    raise NameError('HiThere')
NameError: HiThere

```

raise 唯一的参数就是要触发的异常。这个参数必须是异常实例或异常类（派生自 BaseException 类，例如 Exception 或其子类）。如果传递的是异常类，将通过调用没有参数的构造函数来隐式实例化：

```
raise ValueError  # 'raise ValueError()' 的简化

```

如果只想判断是否触发了异常，但并不打算处理该异常，则可以使用更简单的 raise 语句重新触发异常： >>>

```
>>> try:
...     raise NameError('HiThere')
... except NameError:
...     print('An exception flew by!')
```

```
...     raise
...
An exception flew by!
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise NameError('HiThere')
NameError: HiThere

```

8.5. 异常链

如果一个未处理的异常发生在 except 部分内，它将会有被处理的异常附加到它上面，并包括在错误信息中:>>>

```
>>> try:
...     open("database.sqlite")
... except OSError:
...     raise RuntimeError("unable to handle error")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    open("database.sqlite")
    ~~~~^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'database.sqlite'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError("unable to handle error")
RuntimeError: unable to handle error

```

为了表明一个异常是另一个异常的直接后果， raise 语句允许一个可选的 from 子句:

```
# exc 必须为异常实例或为 None。
raise RuntimeError from exc

```

转换异常时，这种方式很有用。例如：>>>

```
>>> def func():
...     raise ConnectionError
...
>>> try:
...     func()
... except ConnectionError as exc:
...     raise RuntimeError('Failed to open database') from exc
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    func()
    ~~~~^^
  File "<stdin>", line 2, in func
ConnectionError

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError('Failed to open database') from exc
RuntimeError: Failed to open database

```

它还允许使用 `from None` 表达禁用自动异常链:`>>>`

```
>>> try:
...     open('database.sqlite')
... except OSError:
...     raise RuntimeError from None
...
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError from None
RuntimeError

```

异常链机制详见 内置异常。

8.6. 用户自定义异常

程序可以通过创建新的异常类命名自己的异常（Python 类的内容详见 类）。不论是以直接还是间接的方式，异常都应从 Exception 类派生。
异常类可以被定义成能做其他类所能做的任何事，但通常应当保持简单，它往往只提供一些属性，允许相应的异常处理程序提取有关错误的信息。
大多数异常命名都以 “Error” 结尾，类似标准异常的命名。
许多标准模块定义了自己的异常，以报告他们定义的函数中可能出现的错误。

8.7. 定义清理操作

try 语句还有一个可选子句，用于定义在所有情况下都必须要执行的清理操作。例如：>>>

```
>>> try:
...     raise KeyboardInterrupt
... finally:
...     print('Goodbye, world!')
...
Goodbye, world!
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise KeyboardInterrupt
KeyboardInterrupt

```

如果存在 finally 子句，则 finally 子句是 try 语句结束前执行的最后一项任务。不论 try 语句是否触发异常，都会执行 finally 子句。以下内容介绍了几种比较复杂的触发异常情景：

- 如果执行

try 子句期间触发了某个异常，则某个 except 子句应处理该异常。如果该异常没有 except 子句处理，在 finally 子句执行后会被重新触发。

- except

或 else 子句执行期间也会触发异常。 同样，该异常会在 finally 子句执行之后被重新触发。

- 如果

finally 子句中包含 break、continue 或 return 等语句，异常将不会被重新引发。

- 如果执行

try 语句时遇到 break,、continue 或 return 语句，则 finally 子句在执行 break、continue 或 return 语句之前执行。

- 如果

finally 子句中包含 return 语句，则返回值来自 finally 子句的某个 return 语句的返回值，而不是来自 try 子句的 return 语句的返回值。
例如：>>>

```
>>> def bool_return():
...     try:
...         return True
...     finally:
...         return False
...
>>> bool_return()
False

```

这是一个比较复杂的例子：`>>>`

```
>>> def divide(x, y):
...     try:
...         result = x / y
...     except ZeroDivisionError:
...         print("division by zero!")
...     else:
...         print("result is", result)
...     finally:
...         print("executing finally clause")
...
>>> divide(2, 1)
result is 2.0
executing finally clause
>>> divide(2, 0)
division by zero!
executing finally clause
>>> divide("2", "1")
executing finally clause
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    divide("2", "1")
    ~~~~~~^^^^^^^^^^
  File "<stdin>", line 3, in divide
    result = x / y
             ~~^~~
TypeError: unsupported operand type(s) for /: 'str' and 'str'

```

如上所示，任何情况下都会执行 finally 子句。except 子句不处理两个字符串相除触发的 TypeError，因此会在 finally 子句执行后被重新触发。
在实际应用程序中，finally 子句对于释放外部资源（例如文件或者网络连接）非常有用，无论是否成功使用资源。

8.8. 预定义的清理操作

某些对象定义了不需要该对象时要执行的标准清理操作。无论使用该对象的操作是否成功，都会执行清理操作。比如，下例要打开一个文件，并输出文件内容：

```
for line in open("myfile.txt"):
    print(line, end="")

```

这个代码的问题在于，执行完代码后，文件在一段不确定的时间内处于打开状态。在简单脚本中这没有问题，但对于较大的应用程序来说可能会出问题。with 语句支持以及时、正确的清理的方式使用文件对象：

```
with open("myfile.txt") as f:
    for line in f:
        print(line, end="")

```

语句执行完毕后，即使在处理行时遇到问题，都会关闭文件 **f**。和文件一样，支持预定义清理操作的对象会在文档中指出这一点。

8.9. 引发和处理多个不相关的异常

在有些情况下，有必要报告几个已经发生的异常。这通常是在并发框架中当几个任务并行失败时的情况，但也有其他的用例，有时需要是继续执行并收集多个错误而不是引发第一个异常。
内置的 ExceptionGroup 打包了一个异常实例的列表，这样它们就可以一起被引发。它本身就是一个异常，所以它可以像其他异常一样被捕获。>>>

```
>>> def f():
...     excs = [OSError('error 1'), SystemError('error 2')]
...     raise ExceptionGroup('there were problems', excs)
...
>>> f()
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 1, in <module>
  |     f()
```

```
  |     ~^^
  |   File "<stdin>", line 3, in f
  |     raise ExceptionGroup('there were problems', excs)
  | ExceptionGroup: there were problems (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | OSError: error 1
    +---------------- 2 ----------------
    | SystemError: error 2
    +------------------------------------
>>> try:
...     f()
... except Exception as e:
...     print(f'caught {type(e)}: e')
...
caught <class 'ExceptionGroup'>: e
>>>

```

通过使用 `except*` 代替 `except` ，我们可以有选择地只处理组中符合某种类型的异常。在下面的例子中，显示了一个嵌套的异常组，每个 `except*` 子句都从组中提取了某种类型的异常，而让所有其他的异常传播到其他子句，并最终被重新引发。`>>>`

```
>>> def f():
...     raise ExceptionGroup(
...         "group1",
...         [
...             OSError(1),
...             SystemError(2),
...             ExceptionGroup(
...                 "group2",
...                 [
...                     OSError(3),
...                     RecursionError(4)
...                 ]
...             )
...         ]
...     )
...
>>> try:
...     f()
```

```
... except* OSError as e:
...     print("There were OSErrors")
... except* SystemError as e:
...     print("There were SystemErrors")
...
There were OSErrors
There were SystemErrors
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 2, in <module>
  |     f()
  |     ~^^
  |   File "<stdin>", line 2, in f
  |     raise ExceptionGroup(
  |     ...<12 lines>...
  |     )
  | ExceptionGroup: group1 (1 sub-exception)
  +-+---------------- 1 ----------------
    | ExceptionGroup: group2 (1 sub-exception)
    +-+---------------- 1 ----------------
      | RecursionError: 4
      +------------------------------------
>>>

```

注意，嵌套在一个异常组中的异常必须是实例，而不是类型。这是因为在实践中，这些异常通常是那些已经被程序提出并捕获的异常，其模式如下:>>>

```
>>> excs = []
... for test in tests:
...     try:
...         test.run()
...     except Exception as e:
...         excs.append(e)
...
>>> if excs:
...    raise ExceptionGroup("Test Failures", excs)
...

```

8.10. 用注释细化异常情况

当一个异常被创建以引发时，它通常被初始化为描述所发生错误的信息。在有些情况下，在异常被捕获后添加信息是很有用的。为了这个目的，异常有一个 add_note(note) 方法接受一个字符串，并将其添加到异常的注释列表。标准的回溯在异常之后按照它们被添加的顺序呈现包括所有的注释。 >>>

```
>>> try:
...     raise TypeError('bad type')
... except Exception as e:
...     e.add_note('Add some information')
...     e.add_note('Add some more information')
...     raise
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise TypeError('bad type')
TypeError: bad type
Add some information
Add some more information
>>>

```

例如，当把异常收集到一个异常组时，我们可能想为各个错误添加上下文信息。在下文中，组中的每个异常都有一个说明，指出这个错误是什么时候发生的。 >>>

```
>>> def f():
...     raise OSError('operation failed')
...
>>> excs = []
>>> for i in range(3):
...     try:
...         f()
...     except Exception as e:
...         e.add_note(f'Happened in Iteration {i+1}')
...         excs.append(e)
...
>>> raise ExceptionGroup('We have some problems', excs)
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 1, in <module>
  |     raise ExceptionGroup('We have some problems', excs)
```

```
  | ExceptionGroup: We have some problems (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
    |     raise OSError('operation failed')
    | OSError: operation failed
    | Happened in Iteration 1
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
    |     raise OSError('operation failed')
    | OSError: operation failed
    | Happened in Iteration 2
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
    |     raise OSError('operation failed')
    | OSError: operation failed
    | Happened in Iteration 3
    +------------------------------------
>>>
```

9. 类

类提供了把数据和功能绑定在一起的方法。创建新类时创建了新的对象 **类型**，从而能够创建该类型的新 **实例**。实例具有能维持自身状态的属性，还具有能修改自身状态的方法（由其所属的类来定义）。和其他编程语言相比，Python 的类只使用了很少的新语法和语义。Python 的类有点类似于 C++ 和 Modula-3 中类的结合体，而且支持面向对象编程（OOP）的所有标准特性：类的继承机制支持多个基类、派生的类能覆盖基类的方法、类的方法能调用基类中的同名方法。对象可包含任意数量和类型的数据。和模块一样，类也支持 Python 动态特性：在运行时创建，创建后还可以修改。

如果用 C++ 术语来描述的话，类成员（包括数据成员）通常为 **public** （例外的情况见下文 私有变量），所有成员函数都为 **virtual** 。与 Modula-3 中一样，没有用于从对象的方法中引用本对象成员的简写形式：方法函数在声明时，有一个显式的第一个参数代表本对象，该参数由方法调用隐式提供。与在 Smalltalk 中一样，Python 的类也是对象，这为导入和重命名提供了语义支持。与 C++ 和 Modula-3 不同，Python 的内置类型可以用作基类，供用户扩展。此外，与 C++ 一样，具有特殊语法的内置运算符（算术运算符、下标等）都可以为类实例重新定义。
由于缺乏关于类的公认术语，本章中偶尔会使用 Smalltalk 和 C++ 的术语。本章还会使用 Modula-3 的术语，Modula-3 的面向对象语义比 C++ 更接近 Python，但估计听说过这门语言的读者很少。

9.1. 名称和对象

对象之间相互独立，多个名称（甚至是多个作用域内的多个名称）可以绑定到同一对象。这在其他语言中通常被称为别名。Python 初学者通常不容易理解这个概念，处理数字、字符串、元组等不可变基本类型时，可以不必理会。但是，对于涉及可变对象（如列表、字典，以及大多数其他类型）的 Python 代码的语义，别名可能会产生意料之外的效果。这样做，通常是为了让程序受益，因为别名在某些方面就像指针。例如，传递对象的代价很小，因为实现只传递一个指针；如果函数修改了作为参数传递的对象，调用者就可以看到更改——无需像 Pascal 那样用两个不同的机制来传参。

9.2. Python 作用域和命名空间

在介绍类前，首先要介绍 Python 的作用域规则。类定义对命名空间有一些巧妙的技巧，了解作用域和命名空间的工作机制有利于加强对类的理解。并且，即便对于高级 Python 程序员，这方面的知识也很有用。
接下来，我们先了解一些定义。
namespace （命名空间）是从名称到对象的映射。现在，大多数命名空间都使用 Python 字典实现，但除非涉及到性能优化，我们一般不会关注这方面的事情，而且将来也可能会改变这种方式。命名空间的例子有：内置名称集合（包括 abs() 函数以及内置异常的名称等）；一个模块的全局名称；一个函数调用中的局部名称。对象的属性集合也是命名空间的一种形式。关于命名空间的一个重要知识点是，不同命名空间中的名称之间绝对没有关系；例如，两个不同的模块都可以定义 maximize 函数，且不会造成混淆。用户使用函数时必须要在函数名前面加上模块名。
点号之后的名称是 **属性**。例如，表达式 z.real 中，real 是对象 z 的属性。严格来说，对模块中名称的引用是属性引用：表达式 modname.funcname 中，modname 是模块对象，funcname 是模块的属性。模块属性和模块中定义的全局名称之间存在直接的映射：它们共享相同的命名空间！ [1]
属性可以是只读的或者可写的。 在后一种情况下，可以对属性进行赋值。 模块属性是可写的：你可以写入 modname.the_answer = 42 。 也可以使用 del 语句删除可写属性。 例如，
del modname.the_answer 将从名为 modname 对象中移除属性 the_answer。
命名空间是在不同时刻创建的，且拥有不同的生命周期。内置名称的命名空间是在 Python 解释器启动时创建的，永远不会被删除。模块的全局命名空间在读取模块定义时创建；通常，模块的命名空间

也会持续到解释器退出。从脚本文件读取或交互式读取的，由解释器顶层调用执行的语句是 __main__ 模块调用的一部分，也拥有自己的全局命名空间。内置名称实际上也在模块里，即 builtins 。

函数的局部命名空间在函数被调用时被创建，并在函数返回或抛出未在函数内被处理的异常时，被删除。（实际上，用“遗忘”来描述实际发生的情况会更好一些。）当然，每次递归调用都有自己的局部命名空间。

一个命名空间的 **作用域** 是 Python 代码中的一段文本区域，从这个区域可直接访问该命名空间。“可直接访问”的意思是，该文本区域内的名称在被非限定引用时，查找名称的范围，是包括该命名空间在内的。

作用域虽然是被静态确定的，但会被动态使用。执行期间的任何时刻，都会有 3 或 4 个“命名空间可直接访问”的嵌套作用域：

- 最内层作用域，包含局部名称，并首先在其中进行搜索
- 那些外层闭包函数的作用域，包含“非局部、非全局”的名称，从最靠内层的那个作用域开始，逐层向外搜索。
- 倒数第二层作用域，包含当前模块的全局名称
- 最外层（最后搜索）的作用域，是内置名称的命名空间

如果一个名称被声明为全局，则所有引用和赋值都将直接指向“倒数第二层作用域”，即包含模块的全局名称的作用域。 要重新绑定在最内层作用域以外找到的变量，可以使用 nonlocal 语句；如果未使用 nonlocal 声明，这些变量将为只读（尝试写入这样的变量将在最内层作用域中创建一个 **新的** 局部变量，而使得同名的外部变量保持不变）。

通常，当前局部作用域将（按字面文本）引用当前函数的局部名称。在函数之外，局部作用域引用与全局作用域一致的命名空间：模块的命名空间。 类定义在局部命名空间内再放置另一个命名空间。

划重点，作用域是按字面文本确定的：模块内定义的函数的全局作用域就是该模块的命名空间，无论该函数从什么地方或以什么别名被调用。另一方面，实际的名称搜索是在运行时动态完成的。但是，Python 正在朝着“编译时静态名称解析”的方向发展，因此不要过于依赖动态名称解析！（局部变量已经是被静态确定了。）

Python 有一个特殊规定。如果不存在生效的 global 或 nonlocal 语句，则对名称的赋值总是会进入最内层作用域。赋值不会复制数据，只是将名称绑定到对象。删除也是如此：语句 `del x` 从局部作用域引用的命名空间中移除对 `x` 的绑定。所有引入新名称的操作都是使用局部作用域：尤其是 import 语句和函数定义会在局部作用域中绑定模块或函数名称。

global 语句用于表明特定变量在全局作用域里，并应在全局作用域中重新绑定；nonlocal 语句表明特定变量在外层作用域中，并应在外层作用域中重新绑定。

9.2.1. 作用域和命名空间示例

下例演示了如何引用不同作用域和名称空间，以及 global 和 nonlocal 对变量绑定的影响：

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
print("In global scope:", spam)

```

示例代码的输出是：

```
After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam

```

注意，**局部** 赋值（这是默认状态）不会改变 **scope_test** 对 **spam** 的绑定。 nonlocal 赋值会改变 **scope_test** 对 **spam** 的绑定，而 global 赋值会改变模块层级的绑定。
而且，global 赋值前没有 **spam** 的绑定。

9.3. 初探类

类引入了一点新语法，三种新的对象类型和一些新语义。

9.3.1. 类定义语法

最简单的类定义形式如下：

```
class ClassName:
    <语句-1>
    .
    .
    .
    <语句-N>

```

与函数定义 (def 语句) 一样，类定义必须先执行才能生效。把类定义放在 if 语句的分支里或函数内部试试。

在实践中，类定义内的语句通常都是函数定义，但也可以是其他语句。这部分内容稍后再讨论。类里的函数定义一般是特殊的参数列表，这是由方法调用的约定规范所指明的 --- 同样，稍后再解释。

当进入类定义时，将创建一个新的命名空间，并将其用作局部作用域 --- 因此，所有对局部变量的赋值都是在这个新命名空间之内。 特别的，函数定义会绑定到这里的新函数名称。

当 (从结尾处) 正常离开类定义时，将创建一个 **类对象**。 这基本上是一个围绕类定义所创建的命名空间的包装器；我们将在下一节中了解有关类对象的更多信息。 原始的 (在进入类定义之前有效的) 作用域将重新生效，类对象将在这里与类定义头所给出的类名称进行绑定 (在这个示例中为 ClassName)。

9.3.2. Class 对象

类对象支持两种操作：属性引用和实例化。

属性引用 使用 Python 中所有属性引用所使用的标准语法: obj.name。 有效的属性名称是类对象被创建时存在于类命名空间中的所有名称。 因此，如果类定义是这样的:

```
class MyClass:
    """一个简单的示例类"""
    i = 12345

    def f(self):
        return 'hello world'

```

那么 MyClass.i 和 MyClass.f 就是有效的属性引用，将分别返回一个整数和一个函数对象。 类属性也可以被赋值，因此可以通过赋值来改变 MyClass.i 的值。 __doc__ 也是一个有效的属性，将返回所属类的文档字符串: "A simple example class"。

类的 **实例化** 使用函数表示法。 可以把类对象视为是返回该类的一个新实例的不带参数的函数。 举例来说（假设使用上述的类）：

```
x = MyClass()

```

创建类的新 **实例** 并将此对象分配给局部变量 x。
实例化操作（“调用”类对象）会创建一个空对象。 许多类都希望创建的对象实例是根据特定初始状态定制的。 因此一个类可能会定义名为 __init__() 的特殊方法，就像这样:

```
def __init__(self):
    self.data = []

```

当一个类定义了 __init__() 方法时，类的实例化会自动为新创建的类实例唤起 __init__()。 因此在这个例子中，可以通过以下语句获得一个已初始化的新实例:

```
x = MyClass()

```

当然，__init__() 方法还有一些参数用于实现更高的灵活性。 在这种情况下，提供给类实例化运算符的参数将被传递给 __init__()。 例如，>>>

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)

```

9.3.3. 实例对象

现在我们能用实例对象做什么？ 实例对象所能理解的唯一操作是属性引用。 有两种有效的属性名称：数据属性和方法。
数据属性 对应于 Smalltalk 中的“实例变量”，以及 C++ 中的“数据成员”。 数据属性不需要声明；就像局部变量一样，它们将在首次被赋值时产生。 举例来说，如果 x 是上面创建的 MyClass 的实例，则以下代码将打印数值 16，且不保留任何追踪信息:

```
x.counter = 1
while x.counter < 10:
    x.counter = x.counter * 2
print(x.counter)
del x.counter

```

另一种实例属性引用称为 **方法**。 方法是“从属于”对象的函数。
实例对象的有效方法名称依赖于其所属的类。 根据定义，一个类中所有是函数对象的属性都是定义了其实例的相应方法。 因此在我们的示例中，x.f 是有效的方法引用，因为 MyClass.f 是一个函数，而 x.i 不是方法，因为 MyClass.i 不是函数。 但是 x.f 与 MyClass.f 并不是一回事 --- 它是一个 **方法对象**，不是函数对象。

9.3.4. 方法对象

通常，方法在绑定后立即被调用:

```
x.f()

```

在 MyClass 示例中，这将返回字符串 'hello world'。 但是，方法并不是必须立即调用: x.f 是一个方法对象，它可以被保存起来以后再调用。 例如:

```
xf = x.f
while True:
    print(xf())

```

将持续打印 hello world，直到结束。
当一个方法被调用时究竟会发生什么？ 你可能已经注意到尽管 f() 的函数定义指定了一个参数，但上面调用 x.f() 时却没有带参数。 这个参数发生了什么事？ 当一个需要参数的函数在不附带任何参数的情况下被调用时 Python 肯定会引发异常 --- 即使参数实际上没有被使用...
实际上，你可能已经猜到了答案：方法的特殊之处就在于实例对象会作为函数的第一个参数被传入。 在我们的示例中，调用 x.f() 其实就相当于 MyClass.f(x)。 总之，调用一个具有 **n** 个参数的方法就相当于调用再多一个参数的对应函数，这个参数值为方法所属实例对象，位置在其他参数之前。
总而言之，方法的运作方式如下。 当一个实例的非数据属性被引用时，将搜索该实例所属的类。 如果名称表示一个属于函数对象的有效类属性，则指向实例对象和函数对象的引用将被打包为一个方法对象。 当传入一个参数列表调用该方法对象时，将基于实例对象和参数列表构造一个新的参数列表，并传入这个新参数列表调用相应的函数对象。

9.3.5. 类和实例变量

一般来说，实例变量用于每个实例的唯一数据，而类变量用于类的所有实例共享的属性和方法:

```
class Dog:

    kind = 'canine'         # 类变量被所有实例所共享

    def __init__(self, name):
        self.name = name    # 实例变量为每个实例所独有

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.kind                  # 被所有的 Dog 实例所共享
'canine'
>>> e.kind                  # 被所有的 Dog 实例所共享
'canine'
>>> d.name                  # 为 d 所独有
'Fido'
>>> e.name                  # 为 e 所独有
'Buddy'

```

正如 名称和对象 中已讨论过的，共享数据可能在涉及 mutable 对象例如列表和字典的时候导致令人惊讶的结果。 例如以下代码中的 **tricks** 列表不应该被用作类变量，因为所有的 **Dog** 实例将只共享一个单独的列表:

```
class Dog:

    tricks = []             # 类变量的错误用法

    def __init__(self, name):
        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
```

```
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks                # 非预期地被所有的 Dog 实例所共享
['roll over', 'play dead']

```

正确的类设计应该使用实例变量:

```
class Dog:

    def __init__(self, name):
        self.name = name
        self.tricks = []    # 为每个 Dog 实例新建一个空列表

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> e.tricks
['play dead']

```

9.4. 补充说明

如果同样的属性名称同时出现在实例和类中，则属性查找会优先选择实例:>>>

```
>>> class Warehouse:
...     purpose = 'storage'
...     region = 'west'
...
>>> w1 = Warehouse()
>>> print(w1.purpose, w1.region)
storage west
>>> w2 = Warehouse()
```

```
>>> w2.region = 'east'
>>> print(w2.purpose, w2.region)
storage east

```

数据属性可以被方法以及一个对象的普通用户（“客户端”）所引用。 换句话说，类不能用于实现纯抽象数据类型。 实际上，在 Python 中没有任何东西能强制隐藏数据 --- 它是完全基于约定的。 （而在另一方面，用 C 语言编写的 Python 实现则可以完全隐藏实现细节，并在必要时控制对象的访问；此特性可以通过用 C 编写 Python 扩展来使用。）

客户端应当谨慎地使用数据属性 --- 客户端可能通过直接操作数据属性的方式破坏由方法所维护的固定变量。 请注意客户端可以向一个实例对象添加他们自己的数据属性而不会影响方法的可用性，只要保证避免名称冲突 --- 再次提醒，在此使用命名约定可以省去许多令人头痛的麻烦。

在方法内部引用数据属性（或其他方法！）并没有简便方式。 我发现这实际上提升了方法的可读性：当浏览一个方法代码时，不会存在混淆局部变量和实例变量的机会。

方法的第一个参数常常被命名为 self。 这也不过就是一个约定: self 这一名称在 Python 中绝对没有特殊含义。 但是要注意，不遵循此约定会使得你的代码对其他 Python 程序员来说缺乏可读性，而且也可以想像一个 **类浏览器** 程序的编写可能会依赖于这样的约定。

任何一个作为类属性的函数都为该类的实例定义了一个相应方法。 函数定义的文本并非必须包含于类定义之内：将一个函数对象赋值给一个局部变量也是可以的。 例如:

```
# 在类之外定义的函数
def f1(self, x, y):
    return min(x, x+y)

class C:
    f = f1

    def g(self):
        return 'hello world'

    h = g

```

现在 f、g 和 h 都 C 类的指向函数对象的属性，因此它们都是 C 实例的方法 --- 其中 h 与 g 完全等价。 但请注意这种做法通常只会使程序的阅读者感到迷惑。

方法可以通过使用 self 参数的方法属性调用其他方法:

```
class Bag:
    def __init__(self):
        self.data = []
```

```

    def add(self, x):
        self.data.append(x)

    def addtwice(self, x):
        self.add(x)
        self.add(x)

```

方法可以通过与普通函数相同的方式引用全局名称。与方法相关联的全局作用域就是包含该方法的定义语句的模块。（类永远不会被用作全局作用域。）尽管一个人很少会有好的理由在方法中使用全局作用域中的数据，全局作用域依然存在许多合理的使用场景：举个例子，导入到全局作用域的函数和模块可以被方法所使用，定义在全局作用域中的函数和类也一样。通常，包含该方法的类本身就定义在全局作用域中，而在下一节中我们将会发现，为何有些时候方法需要引用其所属类。
每个值都是一个对象，因此具有 **类** （也称为 **类型**），并存储为 object.__class__ 。

9.5. 继承

当然，如果不支持继承，语言特性就不值得称为“类”。派生类定义的语法如下所示:

```
class DerivedClassName(BaseClassName):
    <语句-1>
    .
    .
    .
    <语句-N>

```

名称 BaseClassName 必须定义于可从包含所派生的类的定义的作用域访问的命名空间中。 作为基类名称的替代，也允许使用其他任意表达式。 例如，当基类定义在另一个模块中时，这就会很有用处:

```
class DerivedClassName(modname.BaseClassName):

```

派生类定义的执行过程与基类相同。 当构造类对象时，基类会被记住。 此信息将被用来解析属性引用：如果请求的属性在类中找不到，搜索将转往基类中进行查找。 如果基类本身也派生自其他某个类，则此规则将被递归地应用。
派生类的实例化没有任何特殊之处: DerivedClassName() 会创建该类的一个新实例。 方法引用将按以下方式解析：搜索相应的类属性，如有必要将按基类继承链逐步向下查找，如果产生了一个函数对象则方法引用就生效。

派生类可能会重写其基类的方法。 因为方法在调用同一对象的其他方法时没有特殊权限，所以基类方法在尝试调用调用同一基类中定义的另一方法时，可能实际上调用是该基类的派生类中定义的方法。（对 C++ 程序员的提示：Python 中所有的方法实际上都是 virtual 方法。）

在派生类中的重写方法实际上可能想要扩展而非简单地替换同名的基类方法。 有一种方式可以简单地直接调用基类方法：即调用 BaseClassName.methodname(self, arguments)。 有时这对客户端来说也是有用的。 （请注意仅当此基类可在全局作用域中以 BaseClassName 的名称被访问时方可使用此方式。）

Python有两个内置函数可被用于继承机制：

- 使用

isinstance() 来检查一个实例的类型: isinstance(obj, int) 仅会在 obj.__class__ 为 int 或某个派生自 int 的类时为 True。

- 使用

issubclass() 来检查类的继承关系: issubclass(bool, int) 为 True，因为 bool 是 int 的子类。 但是，issubclass(float, int) 为 False，因为 float 不是 int 的子类。

9.5.1. 多重继承

Python 也支持一种多重继承。 带有多个基类的类定义语句如下所示:

```
class DerivedClassName(Base1, Base2, Base3):
    <语句-1>
    .
    .
    .
    <语句-N>

```

对于多数目的来说，在最简单的情况下，你可以认为搜索从父类所继承属性的操作是深度优先、从左到右的，当层次结构存在重叠时不会在同一个类中搜索两次。 因此，如果某个属性在 DerivedClassName 中找不到，就会在 Base1 中搜索它，然后（递归地）在 Base1 的基类中搜索，如果在那里也找不到，就将在 Base2 中搜索，依此类推。

真实情况比这个更复杂一些；方法解析顺序会动态改变以支持对 super() 的协同调用。 这种方式在某些其他多重继承型语言中被称为后续方法调用，它比单继承型语言中的 super 调用更强大。

动态调整顺序是有必要的，因为所有多重继承的情况都会显示出一个或更多的菱形关联（即至少有一个上级类可通过多条路径被最底层的类所访问）。 例如，所有类都是继承自 object，因此任何多重继承的情况都提供了一条以上的路径可以通向 object。 为了确保基类不会被访问一次以上，动态算法会用一种特殊方式将搜索顺序线性化，保留每个类所指定的从左至右的顺序，只调用每个上级类一次，并且保持单调性（即一个类可以被子类化而不影响其父类的优先顺序）。 总而言之，这些特性使得设

计具有多重继承的可靠且可扩展的类成为可能。 要了解更多细节，请参阅 Python 2.3 方法解析顺序。

9.6. 私有变量

那种仅限从一个对象内部访问的“私有”实例变量在 Python 中并不存在。 但是，大多数 Python 代码都遵循这样一个约定：带有一个下划线的名称 (例如 _spam) 应该被当作是 API 的非公有部分 (无论它是函数、方法或是数据成员)。 这应当被视为一个实现细节，可能不经通知即加以改变。

由于存在对于类私有成员的有效使用场景（例如避免名称与子类所定义的名称相冲突），因此存在对此种机制的有限支持，称为 **名称改写**。 任何形式为 __spam 的标识符（至少带有两个前缀下划线，至多一个后缀下划线）的文本将被替换为 _classname__spam，其中 classname 为去除了前缀下划线的当前类名称。 这种改写不考虑标识符的句法位置，只要它出现在类定义内部就会进行。

参见

私有名称调整规范说明 了解相关详情和特例。

名称改写有助于让子类重写方法而不破坏类内方法调用。例如:

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.__update(iterable)

    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)

    __update = update   # 原始 update() 方法的私有副本

class MappingSubclass(Mapping):

    def update(self, keys, values):
        # 为 update() 提供了新的签名
        # 但不会破坏 __init__()
        for item in zip(keys, values):
            self.items_list.append(item)

```

上面的示例即使在 MappingSubclass 引入了一个 __update 标识符的情况下也不会出错，因为它会在 Mapping 类中被替换为 _Mapping__update 而在 MappingSubclass 类中被替换为 _MappingSubclass__update。

请注意，改写规则的设计主要是为了避免意外冲突；访问或修改被视为私有的变量仍然是可能的。这在特殊情况下甚至会很有用，例如在调试器中。
请注意传递给 exec() 或 eval() 的代码不会将唤起类的类名视作当前类；这类似于 global 语句的效果，因此这种效果仅限于同时经过字节码编译的代码。 同样的限制也适用于 getattr(), setattr() 和 delattr()，以及对于 __dict__ 的直接引用。

9.7. 杂项说明

有时具有类似于 Pascal "record" 或 C "struct" 的数据类型是很有用的，将一些带名称的数据项捆绑在一起。 实现这一目标的理想方式是使用 dataclasses:

```
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    dept: str
    salary: int

```

>>>

```
>>> john = Employee('john', 'computer lab', 1000)
>>> john.dept
'computer lab'
>>> john.salary
1000

```

一段期望使用特定抽象数据类型的 Python 代码通常可以通过传入一个模拟了该数据类型的方法的类作为替代。 例如，如果你有一个基于文件对象来格式化某些数据的函数，你可以定义一个带有 read() 和 readline() 方法以便从字典串缓冲区获取数据的类，并将其作为参数传入。
实例方法对象 也具有属性: m.__self__ 就是带有 m() 方法的实例对象，而 m.__func__ 就是该方法所对应的 函数对象。

9.8. 迭代器

到目前为止，您可能已经注意到大多数容器对象都可以使用 for 语句:

```
for element in [1, 2, 3]:
    print(element)
```

```
for element in (1, 2, 3):
    print(element)
for key in {'one':1, 'two':2}:
    print(key)
for char in "123":
    print(char)
for line in open("myfile.txt"):
    print(line, end='')

```

这种访问风格清晰、简洁又方便。 迭代器的使用非常普遍并使得 Python 成为一个统一的整体。 在幕后，for 语句会在容器对象上调用 iter()。 该函数返回一个定义了 __next__() 方法的迭代器对象，此方法将逐一访问容器中的元素。 当元素用尽时，__next__() 将引发 StopIteration 异常来通知终止 for 循环。 你可以使用 next() 内置函数来调用 __next__() 方法；这个例子显示了它的运作方式:>>>

```
>>> s = 'abc'
>>> it = iter(s)
>>> it
<str_iterator object at 0x10c90e650>
>>> next(it)
'a'
>>> next(it)
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    next(it)
StopIteration

```

了解了迭代器协议背后的机制后，就可以轻松地为你的类添加迭代器行为了。 定义 __iter__() 方法用于返回一个带有 __next__() 方法的对象。 如果类已定义了 __next__()，那么 __iter__() 可以简单地返回 self:

```
class Reverse:
    """对一个序列执行反向循环的迭代器。"""
    def __init__(self, data):
        self.data = data
        self.index = len(data)
```

```

    def __iter__(self):
        return self

    def __next__(self):
        if self.index == 0:
            raise StopIteration
        self.index = self.index - 1
        return self.data[self.index]

```

>>>

```
>>> rev = Reverse('spam')
>>> iter(rev)
<__main__.Reverse object at 0x00A1DB50>
>>> for char in rev:
...     print(char)
...
m
a
p
s

```

9.9. 生成器

生成器 是一个用于创建迭代器的简单而强大的工具。 它们的写法类似于标准的函数，但当它们要返回数据时会使用 yield 语句。 每次在生成器上调用 next() 时，它会从上次离开的位置恢复执行（它会记住上次执行语句时的所有数据值）。 一个显示如何非常容易地创建生成器的示例如下:

```
def reverse(data):
    for index in range(len(data)-1, -1, -1):
        yield data[index]

```

>>>

```
>>> for char in reverse('golf'):
...     print(char)
```

```
...
f
l
o
g

```

可以用生成器来完成的任何功能同样可以通用前一节所描述的基于类的迭代器来完成。 但生成器的写法更为紧凑，因为它会自动创建 __iter__() 和 __next__() 方法。
另一个关键特性在于局部变量和执行状态会在每次调用之间自动保存。 这使得该函数相比使用 self.index 和 self.data 这种实例变量的方式更易编写且更为清晰。
除了会自动创建方法和保存程序状态，当生成器终结时，它们还会自动引发 StopIteration。 这些特性结合在一起，使得创建迭代器能与编写常规函数一样容易。

9.10. 生成器表达式

某些简单的生成器可以写成简洁的表达式代码，所用语法类似列表推导式，但外层为圆括号而非方括号。 这种表达式被设计用于生成器将立即被外层函数所使用的情况。 生成器表达式相比完整的生成器更紧凑但较不灵活，相比等效的列表推导式则更为节省内存。
示例:>>>

```
>>> sum(i*i for i in range(10))                # 平方和
285

>>> xvec = [10, 20, 30]
>>> yvec = [7, 5, 3]
>>> sum(x*y for x,y in zip(xvec, yvec))         # 点乘
260

>>> unique_words = set(word for line in page  for word in line.split())

>>> valedictorian = max((student.gpa, student.name) for student in graduates)

>>> data = 'golf'
>>> list(data[i] for i in range(len(data)-1, -1, -1))
['f', 'l', 'o', 'g']
```

10. 标准库简介

10.1. 操作系统接口

os 模块提供了许多与操作系统交互的函数:>>>

```
>>> import os
>>> os.getcwd()      # 返回当前工作目录
'C:\\Python313'
>>> os.chdir('/server/accesslogs')   # 改变当前工作目录
>>> os.system('mkdir today')   # 在系统 shell 中运行 mkdir 命令
0

```

一定要使用 import os 而不是 from os import * 。这将避免内建的 open() 函数被 os.open() 隐式替换掉，因为它们的使用方式大不相同。

内置的 dir() 和 help() 函数可用作交互式辅助工具，用于处理大型模块，如 os:>>>

```
>>> import os
>>> dir(os)
<返回由模块的所有函数组成的列表>
>>> help(os)
<返回根据模块文档字符串创建的详细说明页面>

```

对于日常文件和目录管理任务， shutil 模块提供了更易于使用的更高级别的接口:>>>

```
>>> import shutil
>>> shutil.copyfile('data.db', 'archive.db')
'archive.db'
>>> shutil.move('/build/executables', 'installdir')
'installdir'

```

10.2. 文件通配符

glob 模块提供了一个在目录中使用通配符搜索创建文件列表的函数:>>>

```
>>> import glob
>>> glob.glob('*.py')
['primes.py', 'random.py', 'quote.py']

```

10.3. 命令行参数

一般的工具脚本常常需要处理命令行参数。 这些参数以列表形式存储在 sys 模块的 **argv** 属性中。 举例来说，让我们查看下面的 demo.py 文件:

```
# 文件 demo.py
import sys
print(sys.argv)

```

以下是在命令行中运行 python demo.py one two three 输出的结果:

```
['demo.py', 'one', 'two', 'three']

```

argparse 模块提供了一种更复杂的机制来处理命令行参数。 以下脚本可提取一个或多个文件名，并可选择要显示的行数:

```
import argparse

parser = argparse.ArgumentParser(
    prog='top',
    description='Show top lines from each file')
parser.add_argument('filenames', nargs='+')
parser.add_argument('-l', '--lines', type=int, default=10)
args = parser.parse_args()
print(args)

```

当在通过 python top.py --lines=5 alpha.txt beta.txt 在命令行运行时，该脚本会将 args.lines 设为 5 并将 args.filenames 设为 ['alpha.txt', 'beta.txt']。

10.4. 错误输出重定向和程序终止

sys 模块还具有 **stdin** ， **stdout** 和 **stderr** 的属性。后者对于发出警告和错误消息非常有用，即使在 **stdout** 被重定向后也可以看到它们:>>>

```
>>> sys.stderr.write('Warning, log file not found starting a new one\n')
Warning, log file not found starting a new one

```

终止脚本的最直接方法是使用 `sys.exit()` 。

10.5. 字符串模式匹配

re 模块为高级字符串处理提供正则表达式工具。对于复杂的匹配和操作，正则表达式提供简洁，优化的解决方案:>>>

```
>>> import re
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')
['foot', 'fell', 'fastest']
>>> re.sub(r'(\b[a-z]+) \1', r'\1', 'cat in the the hat')
'cat in the hat'

```

当只需要简单的功能时，首选字符串方法因为它们更容易阅读和调试:>>>

```
>>> 'tea for too'.replace('too', 'two')
'tea for two'

```

10.6. 数学

math 模块提供对用于浮点数学运算的下层 C 库函数的访问:>>>

```
>>> import math
>>> math.cos(math.pi / 4)
0.70710678118654757
>>> math.log(1024, 2)
10.0

```

random 模块提供了进行随机选择的工具:>>>

```
>>> import random
>>> random.choice(['apple', 'pear', 'banana'])
'apple'
>>> random.sample(range(100), 10)   # 无替代的取样
[30, 83, 16, 4, 8, 81, 41, 50, 18, 33]
>>> random.random()    # [0.0, 1.0) 区间的随机浮点数
0.17970987693706186
>>> random.randrange(6)    # 从 range(6) 中随机选取的整数
```

```
4

```

statistics 模块计算数值数据的基本统计属性（均值，中位数，方差等）:>>>

```
>>> import statistics
>>> data = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> statistics.mean(data)
1.6071428571428572
>>> statistics.median(data)
1.25
>>> statistics.variance(data)
1.3720238095238095

```

SciPy项目 <https://scipy.org> 有许多其他模块用于数值计算。

10.7. 互联网访问

有许多模块可用于访问互联网和处理互联网协议。其中两个最简单的 urllib.request 用于从URL检索数据，以及 smtplib 用于发送邮件:>>>

```
>>> from urllib.request import urlopen
>>> with urlopen('http://worldtimeapi.org/api/timezone/etc/UTC.txt') as response:
...     for line in response:
...         line = line.decode()             # 将字节串转换为字符串
...         if line.startswith('datetime'):
...             print(line.rstrip())         # 去除末尾换行符
...
datetime: 2022-01-01T01:36:47.689215+00:00

>>> import smtplib
>>> server = smtplib.SMTP('localhost')
>>> server.sendmail('soothsayer@example.org', 'jcaesar@example.org',
... """To: jcaesar@example.org
... From: soothsayer@example.org
...
... Beware the Ides of March.
... """)
>>> server.quit()
```

```

```

（请注意，第二个示例需要在localhost上运行的邮件服务器。）

10.8. 日期和时间

datetime 模块提供了以简单和复杂的方式操作日期和时间的类。虽然支持日期和时间算法，但实现的重点是有效的成员提取以进行输出格式化和操作。该模块还支持可感知时区的对象。 >>>

```
>>> # 方便地构造和格式化日期值
>>> from datetime import date
>>> now = date.today()
>>> now
datetime.date(2003, 12, 2)
>>> now.strftime("%m-%d-%y. %d %b %Y is a %A on the %d day of %B.")
'12-02-03. 02 Dec 2003 is a Tuesday on the 02 day of December.'

>>> # 日期值支持日历运算
>>> birthday = date(1964, 7, 31)
>>> age = now - birthday
>>> age.days
14368

```

10.9. 数据压缩

常见的数据存档和压缩格式由模块直接支持，包括：zlib, gzip, bz2, lzma, zipfile 和 tarfile。:>>>

```
>>> import zlib
>>> s = b'witch which has which witches wrist watch'
>>> len(s)
41
>>> t = zlib.compress(s)
>>> len(t)
37
>>> zlib.decompress(t)
b'witch which has which witches wrist watch'
>>> zlib.crc32(s)
226805979

```

10.10. 性能测量

一些Python用户对了解同一问题的不同方法的相对性能产生了浓厚的兴趣。 Python提供了一种可以立即回答这些问题的测量工具。

例如，元组封包和拆包功能相比传统的交换参数可能更具吸引力。timeit 模块可以快速演示在运行效率方面一定的优势:>>>

```
>>> from timeit import Timer
>>> Timer('t=a; a=b; b=t', 'a=1; b=2').timeit()
0.57535828626024577
>>> Timer('a,b = b,a', 'a=1; b=2').timeit()
0.54962537085770791

```

与 timeit 的精细粒度级别相反， profile 和 pstats 模块提供了用于在较大的代码块中识别时间关键部分的工具。

10.11. 质量控制

开发高质量软件的一种方法是在开发过程中为每个函数编写测试，并在开发过程中经常运行这些测试。

doctest 模块提供了一个工具，用于扫描模块并验证程序文档字符串中嵌入的测试。测试构造就像将典型调用及其结果剪切并粘贴到文档字符串一样简单。这通过向用户提供示例来改进文档，并且它允许doctest模块确保代码保持对文档的真实:

```
def average(values):
    """计算数字列表的算术平均值

    >>> print(average([20, 30, 70]))
    40.0
    """
    return sum(values) / len(values)

import doctest
doctest.testmod()   # 自动验证嵌入式测试

```

unittest 模块不像 doctest 模块那样易于使用，但它允许在一个单独的文件中维护更全面的测试集:

```python
import unittest

class TestStatisticalFunctions(unittest.TestCase):

    def test_average(self):
        self.assertEqual(average([20, 30, 70]), 40.0)
        self.assertEqual(round(average([1, 5, 7]), 1), 4.3)
        with self.assertRaises(ZeroDivisionError):
            average([])
        with self.assertRaises(TypeError):
            average(20, 30, 70)

unittest.main()  # 从命令行调用时会执行所有测试
```

12. 虚拟环境和包

12.1. 概述

Python应用程序通常会使用不在标准库内的软件包和模块。应用程序有时需要特定版本的库，因为应用程序可能需要修复特定的错误，或者可以使用库的过时版本的接口编写应用程序。

这意味着一个Python安装可能无法满足每个应用程序的要求。如果应用程序A需要特定模块的1.0版本但应用程序B需要2.0版本，则需求存在冲突，安装版本1.0或2.0将导致某一个应用程序无法运行。

这个问题的解决方案是创建一个 virtual environment，一个目录树，其中安装有特定Python版本，以及许多其他包。

然后，不同的应用将可以使用不同的虚拟环境。 要解决先前需求相冲突的例子，应用程序 A 可以拥有自己的 安装了 1.0 版本的虚拟环境，而应用程序 B 则拥有安装了 2.0 版本的另一个虚拟环境。 如果应用程序 B 要求将某个库升级到 3.0 版本，也不会影响应用程序 A 的环境。

12.2. 创建虚拟环境

用于创建和管理虚拟环境的模块是 venv。 venv 将安装运行命令所使用的 Python 版本（即 --version 选项所报告的版本）。 例如，使用 python3.12 执行命令将会安装 3.12 版。

要创建虚拟环境，请确定要放置它的目录，并将 venv 模块作为脚本运行目录路径:

```python
python -m venv tutorial-env

```

这将创建 tutorial-env 目录，如果它不存在的话，并在其中创建包含 Python 解释器副本和各种支持文件的目录。

虚拟环境的常用目录位置是 .venv。 这个名称通常会令该目录在你的终端中保持隐藏，从而避免需要对所在目录进行额外解释的一般名称。 它还能防止与某些工具所支持的 .env 环境变量定义文件发生冲突。

创建虚拟环境后，您可以激活它。

在Windows上，运行:

```
tutorial-env\Scripts\activate

```

在Unix或MacOS上，运行:

```
source tutorial-env/bin/activate

```

（这个脚本是为bash shell编写的。如果你使用 **csh** 或 **fish** shell，你应该改用 activate.csh 或 activate.fish 脚本。）

激活虚拟环境将改变你所用终端的提示符，以显示你正在使用的虚拟环境，并修改环境以使 python 命令所运行的将是已安装的特定 Python 版本。 例如:

```
$ source ~/envs/tutorial-env/bin/activate
(tutorial-env) $ python
Python 3.5.1 (default, May  6 2016, 10:59:36)
  ...
>>> import sys
>>> sys.path
['', '/usr/local/lib/python35.zip', ...,
'~/envs/tutorial-env/lib/python3.5/site-packages']
>>>

```

要撤销激活一个虚拟环境，请输入:

```
deactivate

```

到终端。

12.3. 使用pip管理包

你可以使用一个名为 **pip** 的程序来安装、升级和移除软件包。 默认情况下 pip 将从 Python Package Index 安装软件包。 你可以在你的 web 浏览器中查看 Python Package Index。

pip 有许多子命令: "install", "uninstall", "freeze" 等等。 （请在 安装 Python 模块 指南页查看完整的 pip 文档。）
您可以通过指定包的名称来安装最新版本的包：

```
(tutorial-env) $ python -m pip install novas
Collecting novas
  Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
  Running setup.py install for novas
Successfully installed novas-3.1.1.3

```

您还可以通过提供包名称后跟 == 和版本号来安装特定版本的包：

```
(tutorial-env) $ python -m pip install requests==2.6.0
Collecting requests==2.6.0
  Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0

```

如果你重新运行这个命令，pip 会注意到已经安装了所请求的版本因而不做任何事。 你可以提供不同的版本号来获取相应版本，或者你可以运行 python -m pip install --upgrade 以将软件包升级到最新版本:

```
(tutorial-env) $ python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
  Found existing installation: requests 2.6.0
    Uninstalling requests-2.6.0:
      Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0

```

python -m pip uninstall 后跟一个或多个要从虚拟环境中删除的包所对应的名称。
python -m pip show 将显示有关某个特定包的信息:

```
(tutorial-env) $ python -m pip show requests
---
Metadata-Version: 2.0
Name: requests
```

```
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages
Requires:

```

python -m pip list 将显示所有在虚拟环境中安装的包:

```
(tutorial-env) $ python -m pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)

```

python -m pip freeze 将产生一个类似的已安装包列表，但其输出会使用 python -m pip install 所期望的格式。 一个常见的约定是将此列表放在 requirements.txt 文件中:

```
(tutorial-env) $ python -m pip freeze > requirements.txt
(tutorial-env) $ cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0

```

然后可以将 requirements.txt 提交给版本控制并作为应用程序的一部分提供。然后用户可以使用 install -r 安装所有必需的包:

```
(tutorial-env) $ python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
  ...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
  ...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
  ...
```

```
Installing collected packages: novas, numpy, requests
  Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```